OAVS | TGT COMPUTER SCIENCE

MCQ QUESTION BANK

C Fundamentals — Complete Topic-wise Coverage

282	11	100%
TOTAL MCQS	CHAPTERS	ANSWER KEYS & EXPLANATIONS

Printable Study Material · Syllabus-Strict Coverage · Revision-Oriented

Table of Contents

01	Chapter 1: C Character Set	30 MCQs
02	Chapter 2: Identifiers and Keywords under ANSI C	34 MCQs
03	Chapter 3: Data Types	29 MCQs
04	Chapter 4: Constants	27 MCQs
05	Chapter 5: Type Qualifiers	25 MCQs
06	Chapter 6: Escape Sequences	26 MCQs
07	Chapter 7: Arithmetic Expressions and Built-in Operators	31 MCQs
08	Chapter 8: Preprocessor Directives and Header Files	24 MCQs
09	Chapter 9: Comments	14 MCQs
10	Chapter 10: sizeof Operator	19 MCQs
11	Chapter 11: Steps Involved in Translation of a C Program	23 MCQs

Note: This question bank strictly covers the prescribed "C Fundamentals" syllabus only (Character Set, Identifiers & Keywords, Data Types, Constants, Type Qualifiers, Escape Sequences, Arithmetic Expressions & Operators, Preprocessor Directives, Header Files, Symbolic Constants, Comments, sizeof Operator, and Steps in Translation of a C Program). No topics outside this syllabus (arrays, strings, functions, pointers, loops, structures, etc.) are included.

CHAPTER 01

Chapter 1: C Character Set

Q1 DEFINITION-BASED — C Character Set

What is a 'character set' in the context of the C programming language?

A. A set of fonts used by the compiler

B. The collection of valid symbols that can be used to write a C program

C. A list of all keywords in C

D. A set of escape sequences only

Ans: B

Explanation:
The character set of C is the complete collection of letters, digits, special symbols, and white spaces that the compiler recognizes as valid for writing source code.

Q2 BASIC — C Character Set

Which of the following are the main categories of characters in the C character set?

A. Letters, Digits, Special symbols, White spaces

B. Keywords, Identifiers, Operators, Constants

C. Tokens, Variables, Functions, Statements

D. Vowels, Consonants, Numbers, Symbols

Ans: A

Explanation:
The C character set is broadly classified into letters (A-Z, a-z), digits (0-9), special symbols, and white space characters.

Q3 CONCEPTUAL — C Character Set

In C, are uppercase and lowercase letters treated as the same character?

A. Yes, C is case-insensitive

B. No, C is case-sensitive and treats them differently

C. Only for keywords

D. Only for identifiers, not keywords

Ans: B

Explanation:
C is a case-sensitive language, so 'A' and 'a' are treated as two distinct characters, affecting identifiers and keywords alike.

Q4 FACT-BASED C Character Set

How many digits are included in the C character set?

A. 8 (0-7)

B. 10 (0-9)

C. 16 (0-9, A-F)

D. 9 (1-9)

Ans: B

Explanation:
The C character set includes the ten decimal digits from 0 to 9, used to form numeric constants and as parts of identifiers (not as the first character).

Q5 BASIC C Character Set

Which of these is classified as a 'white space' character in C?

A. Semicolon (;)

B. Blank space, tab, newline

C. Asterisk (*)

D. Underscore (_)

Ans: B

Explanation:
White space characters include blank space, horizontal tab, vertical tab, newline, and form feed; they are used as separators between tokens.

Q6 CONCEPTUAL C Character Set

Why are white space characters important in C source code?

A. They increase execution speed

B. They separate tokens and improve readability without affecting logic

C. They are mandatory between every two characters

D. They are converted to comments automatically

Ans: B

Explanation:
White spaces act as token separators and enhance readability; the compiler generally ignores extra white space without changing the meaning of the program.

Q7 FACT-BASED C Character Set

Which of the following is NOT a special symbol used in the C character set?

A. @ **B.** #

C. $ **D.** %

Ans: A

Explanation:
The symbol '@' is not part of the standard C character set, whereas #, $ (in some implementations), and % are valid special symbols used in C.

Q8 STATEMENT-BASED C Character Set

Statement: 'The C character set consists only of letters and digits.' Is this statement correct?

A. True **B.** False

C. True, only for ANSI C **D.** True, only for identifiers

Ans: B

Explanation:
This is false; the C character set also includes special symbols and white space characters in addition to letters and digits.

Q9 CONCEPTUAL C Character Set

Which special symbol is used to terminate a simple statement in C?

A. Colon (:) **B.** Semicolon (;)

C. Comma (,) **D.** Period (.)

Ans: B

Explanation:
The semicolon (;) is the special symbol used in C to mark the end of a statement, and it is part of the standard character set.

Q10 FACT-BASED C Character Set

Which of the following special symbols is used for pointer declaration and multiplication in C?

A. &
B. *
C. %
D. #

Ans: B

Explanation:
The asterisk (*) symbol serves dual purposes in C: it denotes multiplication in expressions and is also used to declare pointer variables.

Q11 TRICKY C Character Set

Which of the following characters CANNOT appear anywhere in a valid C identifier?

A. Underscore (_)
B. Digit
C. Letter
D. Space

Ans: D

Explanation:
A space is a white space character and cannot appear within an identifier; identifiers must be a continuous sequence of letters, digits, and underscores.

Q12 CONCEPTUAL C Character Set

Which character is commonly used in C to represent 'continuation' between two words in an identifier?

A. Hyphen (-)
B. Underscore (_)
C. Period (.)
D. Colon (:)

Ans: B

Explanation:
The underscore (_) is treated as a valid character in identifiers and is commonly used to join words, e.g., student_name, since hyphens are not permitted.

Q13 FACT-BASED C Character Set

Which of the following is an example of a 'special symbol' in the C character set?

A. A
B. 9
C. +
D. Tab

Ans: C

Explanation:
The plus sign (+) is a special symbol used as an arithmetic operator; letters and digits belong to other categories, and tab is a white space character.

Q14 BASIC C Character Set

What is the total number of alphabets (upper and lower case) included in the C character set?

A. 26
B. 52
C. 36
D. 62

Ans: B

Explanation:
The C character set includes 26 uppercase letters (A-Z) and 26 lowercase letters (a-z), totaling 52 alphabetic characters.

Q15 CONCEPTUAL C Character Set

Which of the following best describes the role of the C character set in program development?

A. It defines the logic of the program
B. It forms the basic building blocks from which all tokens of the language are constructed
C. It is used only for comments
D. It only applies to string constants

Ans: B

Explanation:
The character set provides the fundamental symbols from which keywords, identifiers, constants, operators, and other tokens of the C language are built.

Q16 TRICKY C Character Set

Which of the following is a valid character in C but CANNOT be used as the first character of an identifier?

A. Letter
B. Underscore
C. Digit
D. Both letter and underscore

Ans: C

Explanation:
Digits are valid characters in the C character set and can appear within identifiers, but an identifier must not begin with a digit.

Q17 FACT-BASED C Character Set

Which special symbol in C is used to access the address of a variable?

A. *
B. &
C. %
D. #

Ans: B

Explanation:
The ampersand (&) is the special symbol used as the 'address of' operator to retrieve a variable's memory address.

Q18 APPLICATION-ORIENTED C Character Set

A student writes a program using the symbol '~' for a calculation. To which category of the C character set does '~' belong?

A. Letter
B. Digit
C. Special symbol
D. White space

Ans: C

Explanation:
The tilde (~) is categorized as a special symbol in C, commonly used as the bitwise complement operator.

Q19 COMPARISON-BASED C Character Set

How does a 'digit' differ from a 'letter' in terms of identifier formation in C?

A. Digits can start an identifier, letters cannot

B. Letters can start an identifier, digits cannot start it but can appear later

C. Both can never appear in identifiers

D. Both must always start an identifier

Ans: B

Explanation:
In C, an identifier must begin with a letter or underscore; digits are valid characters in the set but cannot be the first character of an identifier.

Q20 FACT-BASED C Character Set

Which of the following is classified as a 'white space' character along with blank space and tab?

A. Newline

B. Asterisk

C. Hash

D. Underscore

Ans: A

Explanation:
Newline, along with blank space, horizontal tab, and vertical tab, is categorized as a white space character in the C character set.

Q21 CONCEPTUAL C Character Set

Which of the following correctly lists categories under the C character set as per ANSI C?

A. Letters, Digits, Special characters, White spaces

B. Variables, Constants, Keywords, Operators

C. Tokens, Expressions, Statements, Functions

D. Header files, Macros, Comments, Directives

Ans: A

Explanation:
ANSI C classifies its character set into letters, digits, special characters, and white spaces, which together form the basis of all valid C tokens.

Q22 MATCH-THE-FOLLOWING C Character Set

Match the character with its category: (1) Z (2) 7 (3) % (4) Tab — with categories (i) Digit (ii) Special symbol (iii) Letter (iv) White space

A. 1-iii, 2-i, 3-ii, 4-iv

B. 1-i, 2-iii, 3-iv, 4-ii

C. 1-ii, 2-iv, 3-i, 4-iii

D. 1-iv, 2-ii, 3-iii, 4-i

Ans: A

Explanation:
Z is a letter, 7 is a digit, % is a special symbol, and Tab is a white space character — matching each character to its correct category in the C character set.

Q23 FACT-BASED C Character Set

Which of the following symbols is used in C to denote the modulus (remainder) operation as well as being part of the character set?

A. &

B. %

C. @

D. ^

Ans: B

Explanation:
The percent symbol (%) is part of the C special character set and is used as the modulus operator to find the remainder of integer division.

Q24 TRICKY C Character Set

Which of the following is true regarding the character '_' (underscore) in C?

A. It is treated as a special symbol only and cannot be used in identifiers

B. It is treated like a letter and can be used to start or appear within identifiers

C. It can only appear at the end of an identifier

D. It is not part of the C character set

Ans: B

Explanation:
The underscore is treated functionally like a letter in C; it can be the first character of an identifier or appear anywhere within it.

Q25 OUTPUT-BASED C Character Set

Which of the following character sequences contains ONLY valid C character-set members and no space?

A. sum#1

B. sum 1

C. sum@1

D. sum_1

Ans: D

Explanation:
sum_1 contains only a letter, underscore, and a digit, all of which are valid C character-set members without any invalid symbol or space.

Q26 APPLICATION-ORIENTED C Character Set

While typing source code, a programmer presses the Enter key after a statement. Which category of the C character set does this correspond to?

A. Special symbol

B. Digit

C. White space (newline)

D. Letter

Ans: C

Explanation:
Pressing Enter inserts a newline character, which belongs to the white space category in the C character set and is used to separate lines of code.

Q27 COMPARISON-BASED C Character Set

Which of the following pairs correctly contrasts two categories of the C character set?

A. Letters vs Digits — letters form words, digits form numeric constants

B. Keywords vs Identifiers — both are part of the character set directly

C. Comments vs White space — both are character set categories

D. Operators vs Functions — both are basic character categories

Ans: A

Explanation:
Letters and digits are two distinct categories of the character set; letters are primarily used to build identifiers and keywords, while digits build numeric values.

Q28 PREVIOUS-YEAR PATTERN C Character Set

As per ANSI C character set classification, which of the following is grouped under 'special symbols'?

A. + , - , * , /

B. A , B , C

C. 0 , 1 , 2

D. Space, Tab

Ans: A

Explanation:
Arithmetic and other operator symbols such as +, -, *, and / fall under the special symbols category of the ANSI C character set.

Q29 FILL-IN-THE-BLANK C Character Set

The C character set is composed of letters, digits, special symbols, and _____.

A. Comments

B. White spaces

C. Keywords

D. Functions

Ans: B

Explanation:
White spaces, along with letters, digits, and special symbols, complete the four basic categories of the C character set.

Q30 ASSERTION-REASON C Character Set

Assertion (A): The C character set forms the foundation for constructing tokens like keywords and identifiers. Reason (R): Tokens are made up only of special symbols. Choose the correct option.

A. Both A and R are true and R explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: B

Explanation:
The assertion is correct since tokens are built from the character set, but the reason is false because tokens use letters and digits too, not just special symbols.

Chapter 2: Identifiers and Keywords under ANSI C

Q31 DEFINITION-BASED — Identifiers & Keywords

What is an 'identifier' in C programming?

A. A reserved word with predefined meaning

B. A user-defined name given to variables, functions, arrays, and other program elements

C. A special symbol used in expressions

D. A type of constant

Ans: B

Explanation:
An identifier is a name created by the programmer to identify variables, functions, arrays, or other user-defined entities in a C program.

Q32 BASIC — Identifiers & Keywords

Which of the following is a valid rule for naming identifiers in C?

A. Must start with a digit

B. Can contain spaces

C. Must start with a letter or underscore, followed by letters, digits, or underscores

D. Can be a reserved keyword

Ans: C

Explanation:
A valid C identifier must begin with a letter (A-Z, a-z) or underscore, and may be followed by any combination of letters, digits, and underscores, without spaces.

Q33 CONCEPTUAL — Identifiers & Keywords

Why can't an identifier in C be the same as a keyword?

A. Keywords are case-insensitive

B. Keywords are reserved words with predefined meaning to the compiler and cannot be redefined

C. Identifiers must always be longer than keywords

D. There is no such restriction

Ans: B

Explanation:
Keywords have fixed, predefined meanings understood by the compiler; using one as an identifier would create ambiguity, so this is disallowed.

Q34 OUTPUT-BASED Identifiers & Keywords

Which of the following is a VALID identifier in C?

A. 2value
B. value-2
C. _value2
D. value 2

Ans: C

Explanation:
_value2 starts with an underscore followed by letters and a digit, satisfying all naming rules; the others start with a digit, contain a hyphen, or contain a space.

Q35 OUTPUT-BASED Identifiers & Keywords

Which of the following is an INVALID identifier in C?

A. total_marks
B. _temp
C. 3rdValue
D. Sum1

Ans: C

Explanation:
3rdValue is invalid because an identifier cannot begin with a digit; it must start with a letter or underscore.

Q36 FACT-BASED Identifiers & Keywords

How many keywords are defined in standard ANSI C (C89)?

A. 28
B. 30
C. 32
D. 35

Ans: C

Explanation:
Standard ANSI C (C89) defines exactly 32 reserved keywords, each with a specific predefined purpose in the language.

Q37 DEFINITION-BASED Identifiers & Keywords

What is a 'keyword' in C?

A. A user-defined variable name

B. A reserved word that has a fixed, predefined meaning and cannot be used as an identifier

C. Any word longer than 8 characters

D. A type of operator

Ans: B

Explanation:
A keyword is a reserved word in C with a fixed meaning defined by the language itself; it cannot be redefined or used as a variable name.

Q38 FACT-BASED Identifiers & Keywords

Which of the following is a valid ANSI C keyword?

A. integer

B. Float

C. double

D. String

Ans: C

Explanation:
double is a reserved ANSI C keyword used to declare double-precision floating-point variables; the others are either misspelled or not standard keywords.

Q39 TRICKY Identifiers & Keywords

Is 'Float' (with capital F) a valid keyword in C?

A. Yes, keywords are case-insensitive

B. No, because C is case-sensitive and the actual keyword is 'float' in lowercase

C. Yes, but only in ANSI C90

D. No, because keywords cannot have more than 4 letters

Ans: B

Explanation:
Since C is case-sensitive, 'Float' is different from the keyword 'float'; only the lowercase form is recognized as a reserved keyword.

Q40 COMPARISON-BASED Identifiers & Keywords

What is the key difference between an identifier and a keyword?

A. Identifiers are predefined; keywords are user-defined

B. Keywords are predefined and reserved; identifiers are user-defined names

C. Both are exactly the same in C

D. Identifiers cannot contain letters

Ans: B

Explanation:
Keywords are reserved words with fixed meanings built into the language, whereas identifiers are names that programmers create for their own variables and functions.

Q41 CONCEPTUAL Identifiers & Keywords

Which characteristic distinguishes keywords from ordinary identifiers in terms of usage?

A. Keywords can be redefined by the user

B. Keywords always have a predefined purpose and cannot be used as variable names

C. Keywords must always be in uppercase

D. Keywords are optional in a program

Ans: B

Explanation:
A defining characteristic of keywords is that their meaning is fixed by the language specification, so they cannot be repurposed as user-defined identifiers.

Q42 OUTPUT-BASED Identifiers & Keywords

Which of the following identifiers is INVALID due to containing a special character not allowed in identifiers?

A. my_var

B. my-var

C. myvar1

D. _myvar

Ans: B

Explanation:
my-var is invalid because the hyphen (-) is not a permitted character in identifiers; only letters, digits, and underscores are allowed.

Q43 STATEMENT-BASED Identifiers & Keywords

Statement: 'An identifier can have any number of characters, but only the first few are significant depending on the compiler.' Is this generally true for ANSI C?

A. True

B. False

C. True, only for keywords

D. False, identifiers have no length limit at all in practice

Ans: A

Explanation:
ANSI C historically guarantees significance of at least the first 31 characters of internal identifiers; some compilers may have implementation-specific limits beyond that.

Q44 FACT-BASED Identifiers & Keywords

Which of the following is NOT a reserved keyword in ANSI C?

A. return

B. switch

C. main

D. static

Ans: C

Explanation:
'main' is not a reserved keyword; it is simply a conventionally named identifier (function name) recognized as the program's entry point, not a language keyword.

Q45 BASIC Identifiers & Keywords

Which of the following symbols, if used inside an identifier name, would make it invalid?

A. Underscore

B. Digit (not as first character)

C. Letter

D. Dollar sign in strict ANSI C

Ans: D

Explanation:
In strict ANSI C, the dollar sign ($) is not part of the standard identifier character set, although some compilers allow it as an extension.

Q46 CONCEPTUAL Identifiers & Keywords

Can an identifier in C be identical to a predefined library function name like 'printf'?

A. No, it is strictly forbidden by the C standard

B. Technically the compiler may allow redefining it, but it is bad practice since it shadows the standard function

C. Yes, with no consequence at all

D. No, 'printf' is a keyword

Ans: B

Explanation:
printf is a standard library function, not a keyword, so redefining it as an identifier is syntactically possible but is strongly discouraged as it hides the original function.

Q47 FILL-IN-THE-BLANK Identifiers & Keywords

A valid identifier must begin with a letter or a(n) _____.

A. Digit

B. Underscore

C. Special symbol

D. Keyword

Ans: B

Explanation:
Besides letters, an underscore is also a permitted first character for a valid C identifier.

Q48 FACT-BASED Identifiers & Keywords

Which of these is an example of a type qualifier keyword in ANSI C?

A. unsigned

B. main

C. printf

D. value

Ans: A

Explanation:
'unsigned' is a reserved ANSI C keyword used as a type qualifier; 'main' and 'printf' are conventional identifiers, not keywords.

Q49 APPLICATION-ORIENTED Identifiers & Keywords

A programmer names a variable 'int_value' to store an integer. Is this a valid identifier?

A. No, because it contains 'int' which is a keyword

B. Yes, because it only starts with letters and uses an underscore; keywords inside a longer identifier are fine

C. No, identifiers cannot contain underscores

D. No, it exceeds character limits

Ans: B

Explanation:
Although 'int' is a keyword, 'int_value' as a whole is a different, valid identifier since the restriction applies only to using a keyword by itself, not as part of a longer name.

Q50 TRICKY Identifiers & Keywords

Which of the following is true about keywords in ANSI C?

A. They can be used as identifiers if declared with different data types

B. They are fixed in number and meaning, and cannot be altered by the programmer

C. New keywords can be added by the programmer using #define

D. They are case-insensitive across all compilers

Ans: B

Explanation:
ANSI C keywords are a fixed, standardized set with predefined meaning; programmers cannot alter, redefine, or add to this list within the language rules.

Q51 PREVIOUS-YEAR PATTERN Identifiers & Keywords

Which of the following best lists examples of valid ANSI C keywords used for loop and decision control?

A. if, else, for, while

B. Sum, Total, Average, Count

C. value1, value2, result

D. MyFunction, MainProgram

Ans: A

Explanation:
if, else, for, and while are all reserved ANSI C keywords used to implement decision-making and looping control structures.

Q52 OUTPUT-BASED Identifiers & Keywords

Identify the INVALID identifier among the following: 'Total#1', 'Total_1', 'Total1', '_Total'.

A. Total#1

B. Total_1

C. Total1

D. _Total

Ans: A

Explanation:
Total#1 is invalid because the hash symbol (#) is not allowed within an identifier; the other three follow valid identifier-naming rules.

Q53 CONCEPTUAL Identifiers & Keywords

What happens if a programmer tries to use a reserved keyword like 'int' as a variable name?

A. The program runs normally

B. The compiler generates a syntax error since keywords cannot be redefined as identifiers

C. It is automatically converted to a valid identifier

D. It only causes a warning, never an error

Ans: B

Explanation:
Since keywords have fixed syntactic roles in the compiler's grammar, attempting to use one as a variable name results in a compilation error.

Q54 DEFINITION-BASED Identifiers & Keywords

Which statement best defines the 'scope' relevance of an identifier's naming rule regarding case sensitivity?

A. 'Sum' and 'sum' refer to the same identifier

B. 'Sum' and 'sum' are treated as two different identifiers because C is case-sensitive

C. Case sensitivity applies only to keywords, not identifiers

D. Identifiers ignore case but keywords do not

Ans: B

Explanation:
Because C is case-sensitive, identifiers differing only in letter case, such as 'Sum' and 'sum', are treated as completely distinct identifiers.

Q55 FACT-BASED

Identifiers & Keywords

Which of the following is classified as a storage-class-related keyword in ANSI C?

A. auto

B. sizeof

C. char

D. return

Ans: A

Explanation:
'auto' is a storage class keyword in ANSI C, used to specify default automatic storage duration for local variables (though rarely used explicitly).

Q56 COMPARISON-BASED

Identifiers & Keywords

How do 'identifiers' and 'keywords' compare in terms of who defines them?

A. Both are defined by the programmer

B. Identifiers are defined by the programmer; keywords are predefined by the C language standard

C. Both are predefined by the compiler vendor only

D. Keywords are defined by the programmer; identifiers by the standard

Ans: B

Explanation:
Identifiers are created by programmers to name their own program elements, while keywords are fixed and predefined as part of the ANSI C language specification.

Q57 ASSERTION-REASON

Identifiers & Keywords

Assertion (A): 'while' cannot be used as a variable name in C. Reason (R): 'while' is a reserved keyword used for loop control. Choose the correct option.

A. Both A and R are true and R correctly explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: A

Explanation:
Both statements are true: 'while' is indeed a reserved keyword for looping, and this reserved status is exactly why it cannot be used as a variable name.

Q58 TRICKY

Identifiers & Keywords

Which of the following identifier pairs would the C compiler treat as IDENTICAL?

A. Value and value

B. value1 and value1

C. VALUE and Value

D. Sum and sum

Ans: B

Explanation:

Only 'value1' and 'value1' are exactly identical in spelling and case; all other pairs differ in letter case and would be treated as distinct identifiers in case-sensitive C.

Q59 FACT-BASED

Identifiers & Keywords

Which of the following is an ANSI C keyword associated with defining a structure?

A. struct

B. array

C. record

D. object

Ans: A

Explanation:

'struct' is the reserved ANSI C keyword used to define user-defined composite data types known as structures.

Q60 CONCEPTUAL

Identifiers & Keywords

What is the maximum recommended practice regarding identifier length for portability in ANSI C?

A. No more than 1 character

B. Keep significant characters within standard limits (e.g., first 31 characters) for portability across compilers

C. Identifiers must be exactly 8 characters

D. There is no concern about identifier length

Ans: B

Explanation:

Although modern compilers support long identifiers, ANSI C historically guarantees significance only up to a certain number of characters, so keeping names within that limit aids portability.

Q61 APPLICATION-ORIENTED Identifiers & Keywords

A beginner names two variables 'Marks' and 'marks' in the same C program, intending them to represent the same value. What will actually happen?

A. The compiler will merge them into one variable

B. They will be treated as two completely separate identifiers due to case sensitivity

C. It will cause a syntax error immediately

D. C automatically converts the second to lowercase

Ans: B

Explanation:
Since C distinguishes identifiers by case, 'Marks' and 'marks' are treated as two distinct variables, which could lead to logical errors if the programmer intended them to be the same.

Q62 FILL-IN-THE-BLANK Identifiers & Keywords

Keywords in ANSI C are always written in _____ letters.

A. Uppercase

B. Lowercase

C. Mixed case

D. Title case

Ans: B

Explanation:
All standard ANSI C keywords (such as int, float, if, while) are defined and must be used in lowercase letters.

Q63 PREVIOUS-YEAR PATTERN Identifiers & Keywords

Which option correctly lists three valid ANSI C keywords related to data types and type qualifiers together?

A. int, signed, double

B. Number, Decimal, Text

C. value, count, total

D. Main, Print, Input

Ans: A

Explanation:
int and double are basic data type keywords, while signed is a type qualifier keyword; all three are valid reserved words in ANSI C.

Q64 DEFINITION-BASED Identifiers & Keywords

What does it mean to say identifiers are 'case-sensitive' in C?

A. Identifiers must always be typed in capital letters

B. Uppercase and lowercase versions of the same letters are treated as different characters

C. Case sensitivity only matters for keywords

D. It means identifiers cannot use letters at all

Ans: B

Explanation:
Case sensitivity means the compiler distinguishes between uppercase and lowercase letters, so identifiers differing only in case are considered different.

Chapter 3: Data Types

Q65 DEFINITION-BASED — Data Types

What is a 'data type' in C?

A. A keyword used only in loops

B. A classification that specifies the kind of value a variable can hold and the operations that can be performed on it

C. A type of operator

D. A function used for input/output

Ans: B

Explanation:
A data type defines the nature of data (such as integer, floating-point, or character) a variable can store, along with the memory it occupies and valid operations on it.

Q66 BASIC — Data Types

Which of the following are the basic (primary) data types in C?

A. int, float, double, char

B. array, pointer, structure

C. if, else, switch

D. main, printf, scanf

Ans: A

Explanation:
int, float, double, and char are the four fundamental basic data types provided by C to handle integers, floating-point numbers, and characters.

Q67 CONCEPTUAL — Data Types

What is meant by 'derived data types' in C, as introductory concept?

A. Data types directly built into the hardware

B. Data types constructed from the basic data types, such as arrays and pointers

C. Keywords used for input

D. Special symbols only

Ans: B

Explanation:
Derived data types are formed using the basic data types as building blocks; examples include arrays and pointers, which derive their behavior from underlying basic types.

Q68 CLASSIFICATION Data Types

How are data types in C broadly classified?

A. Primary, Derived, and User-defined types

B. Only Primary and Secondary

C. Only Numeric and Non-numeric

D. Only Static and Dynamic

Ans: A

Explanation:
C classifies data types into primary (basic) types, derived types (like arrays and pointers), and user-defined types (like structures and unions).

Q69 FACT-BASED Data Types

Which basic data type in C is used to store a single character?

A. int

B. float

C. char

D. double

Ans: C

Explanation:
The 'char' data type is specifically designed to store a single character, internally represented using its corresponding integer (ASCII) value.

Q70 STORAGE REQUIREMENTS Data Types

On most standard 32-bit systems, how much memory does an 'int' typically occupy?

A. 1 byte

B. 2 bytes

C. 4 bytes

D. 8 bytes

Ans: C

Explanation:
On most modern 32-bit and 64-bit systems, an 'int' typically occupies 4 bytes, although this can vary by compiler and architecture.

Q71 STORAGE REQUIREMENTS Data Types

How much memory does a 'char' data type typically occupy in C?

A. 1 byte
B. 2 bytes
C. 4 bytes
D. 8 bytes

Ans: A

Explanation:
A 'char' variable typically occupies 1 byte of memory, which is sufficient to store a single character's ASCII value.

Q72 STORAGE REQUIREMENTS Data Types

How much memory does a 'double' data type typically occupy on most systems?

A. 4 bytes
B. 8 bytes
C. 2 bytes
D. 16 bytes

Ans: B

Explanation:
A 'double' (double-precision floating point) typically occupies 8 bytes, offering greater precision and range than a 'float'.

Q73 COMPARISON-BASED Data Types

What is the key difference between 'float' and 'double' data types?

A. float stores integers, double stores characters
B. double offers greater precision and typically uses more memory than float
C. They are exactly identical in C
D. float is used only for negative numbers

Ans: B

Explanation:
Both store floating-point numbers, but 'double' provides higher precision and range, generally occupying 8 bytes compared to float's typical 4 bytes.

Q74 CONCEPTUAL Data Types

Why does C require explicit data types for variables?

A. To make the program longer

B. To inform the compiler how much memory to allocate and how to interpret the stored value

C. Because C cannot run without colors

D. It is purely a stylistic choice with no technical reason

Ans: B

Explanation:
Declaring a data type tells the compiler the amount of memory to reserve and the rules for interpreting and operating on the stored data.

Q75 FACT-BASED Data Types

Which of the following is the correct format specifier used with 'printf' for an int variable?

A. %c

B. %f

C. %d

D. %s

Ans: C

Explanation:
%d is the standard format specifier used to print integer values with printf in C.

Q76 FACT-BASED Data Types

Which format specifier is used with 'printf' to display a float value?

A. %d

B. %f

C. %c

D. %ld

Ans: B

Explanation:
%f is used to print floating-point (float) values using printf in standard C.

Q77 BASIC Data Types

Which data type would be most appropriate to store a student's roll number (a whole number)?

A. char
B. float
C. int
D. double

Ans: C

Explanation:
Since a roll number is a whole number without decimal parts, the 'int' data type is the most appropriate and memory-efficient choice.

Q78 APPLICATION-ORIENTED Data Types

Which data type should be used to store a value like 99.95 (a price with decimals)?

A. int
B. char
C. float
D. void

Ans: C

Explanation:
Since the value contains a fractional/decimal part, 'float' (or double) is the appropriate data type to represent it accurately.

Q79 TRICKY Data Types

What will be the likely result if an integer value like 7/2 is computed using int data type in C?

A. 3.5
B. 3
C. 4
D. Error

Ans: B

Explanation:
When both operands are of type int, C performs integer division, truncating the decimal part, so 7/2 evaluates to 3, not 3.5.

Q80 CONCEPTUAL Data Types

Which of the following best explains why 'char' is considered a numeric-like basic data type in C?

A. Characters are stored using their ASCII (integer) codes internally

B. Because char can only store the digit 0-9

C. Because char is the same as int in all respects

D. Characters are stored as binary images, not numbers

Ans: A

Explanation:
Internally, a char is stored as a small integer representing its ASCII code, which is why character variables can participate in arithmetic operations.

Q81 STATEMENT-BASED Data Types

Statement: 'The size of a data type in C is fixed and identical across every compiler and machine.' Is this statement true?

A. True

B. False — size can vary depending on the compiler and machine architecture

C. True, only for char

D. True, only for int

Ans: B

Explanation:
While ANSI C specifies minimum ranges, the exact size of data types like int can vary across different compilers and machine architectures.

Q82 FACT-BASED Data Types

Which of the following data types generally has the LARGEST storage size among the basic types?

A. char

B. int

C. float

D. double

Ans: D

Explanation:
Among the basic data types, 'double' typically occupies the largest memory size (commonly 8 bytes), providing the widest range and precision.

Q83 CLASSIFICATION Data Types

Under which classification would a 'pointer' data type fall, as an introductory derived concept?

A. Primary data type

B. Derived data type

C. User-defined data type

D. Symbolic constant

Ans: B

Explanation:
A pointer is classified as a derived data type since it is built upon a basic type and represents the memory address of a variable of that type.

Q84 CONCEPTUAL Data Types

What determines the range of values a particular data type can store?

A. The compiler's color scheme

B. The number of bytes/bits allocated to that data type

C. The programmer's typing speed

D. The number of variables declared

Ans: B

Explanation:
The range of values a data type can represent is directly determined by how many bits/bytes of memory are allocated to store it.

Q85 FACT-BASED Data Types

Which of these is generally true regarding the relative storage size ordering of basic data types in C (typical case)?

A. char ≤ int ≤ float ≤ double

B. double ≤ float ≤ int ≤ char

C. int ≤ char ≤ double ≤ float

D. float ≤ char ≤ int ≤ double

Ans: A

Explanation:
In typical implementations, char (1 byte) is smallest, followed by int and float (commonly 4 bytes each), with double (8 bytes) generally being the largest.

Q86 PREVIOUS-YEAR PATTERN Data Types

In OAVS-style exams, which of the following correctly identifies all FOUR basic/primitive data types of C together?

A. int, float, double, char

B. int, array, struct, union

C. char, string, boolean, byte

D. float, double, long, short

Ans: A

Explanation:
The four basic/primitive data types specified in the syllabus and standard C are int, float, double, and char.

Q87 APPLICATION-ORIENTED Data Types

A program needs to store whether a light is 'ON' or 'OFF' using the most basic data type approach in classic C (no boolean type). Which is most suitable?

A. float

B. double

C. int (using 0 or 1)

D. char array only

Ans: C

Explanation:
Classic C lacks a dedicated boolean type, so an 'int' is conventionally used with 0 representing false/OFF and 1 representing true/ON.

Q88 TRICKY Data Types

Which statement correctly distinguishes 'basic data types' from 'user-defined data types' in C?

A. Basic types are built into the language; user-defined types like structures are created by the programmer using basic types

B. Both are identical in every respect

C. User-defined types are predefined by ANSI C only

D. Basic types must always be combined with pointers

Ans: A

Explanation:
Basic data types (int, float, char, double) are inherent to the C language, whereas user-defined types such as structures and unions are constructed by programmers using these basics.

Q89 FILL-IN-THE-BLANK Data Types

The data type used to represent 'no value' or absence of a return type for a function in C is _____.

A. int

B. void

C. null

D. empty

Ans: B

Explanation:
'void' is the keyword/data type used in C to indicate the absence of a value, such as for a function that does not return any value.

Q90 CONCEPTUAL Data Types

What is the relationship between a data type and the operations a variable can perform?

A. Data types have no impact on operations

B. Data type determines which operations (e.g., arithmetic on numbers, character handling) are meaningful for a variable

C. All data types support exactly the same operations

D. Operations are determined only by the variable's name

Ans: B

Explanation:
The data type defines the set of valid and meaningful operations; for instance, arithmetic addition is meaningful for int/float but not directly for arbitrary structures.

Q91 COMPARISON-BASED Data Types

Which comparison correctly differentiates 'int' from 'float' regarding the kind of values stored?

A. int stores whole numbers; float stores numbers with fractional/decimal parts

B. int stores characters; float stores whole numbers

C. Both store identical ranges of values

D. float cannot store negative numbers

Ans: A

Explanation:
int is designed to store whole (integer) numbers without decimals, while float is designed to store real numbers that include a fractional component.

Q92 FACT-BASED Data Types

Which keyword in C is used to declare a double-precision floating-point variable?

A. float

B. double

C. long float

D. decimal

Ans: B

Explanation:
The keyword 'double' is used specifically to declare double-precision floating-point variables, offering more precision than 'float'.

Q93 STATEMENT-BASED Data Types

Statement: 'A char variable in C can only store alphabetic letters.' Evaluate this statement.

A. True

B. False — a char can store any single character including digits, symbols, and control characters

C. True, only uppercase letters

D. True, only for ANSI C

Ans: B

Explanation:
A char variable can store any single character from the character set, including letters, digits, punctuation, and special/control characters, not just alphabets.

Chapter 4: Constants

Q94 DEFINITION-BASED Constants

What is a 'constant' in C programming?

A. A value that can change during program execution

B. A fixed value that does not change during the execution of a program

C. A type of variable declared using 'var'

D. A loop control structure

Ans: B

Explanation:
A constant is a fixed value, such as a number or character, that remains unchanged throughout the execution of the program.

Q95 CLASSIFICATION Constants

Which of the following are recognized types of constants in C as per the syllabus?

A. Integer, Floating, Character, String, Symbolic

B. Only Integer and Float

C. Only String constants

D. Boolean and Null constants

Ans: A

Explanation:
C supports integer constants, floating constants, character constants, string constants, and symbolic constants (defined using #define).

Q96 FACT-BASED Constants

Which of the following is a valid example of an integer constant in C?

A. 3.14

B. 'A'

C. 100

D. "Hello"

Ans: C

Explanation:
100 is a whole number without a decimal point or quotes, making it a valid integer constant.

Q97 FACT-BASED Constants

Which of the following is an example of a floating constant in C?

A. 25 **B.** 'x'

C. 12.5 **D.** "text"

Ans: C

Explanation:
12.5 contains a decimal point, which qualifies it as a floating-point (real) constant in C.

Q98 FACT-BASED Constants

Which of the following correctly represents a character constant in C?

A. A **B.** 'A'

C. "A" **D.** A;

Ans: B

Explanation:
A character constant in C is written by enclosing a single character within single quotes, such as 'A'.

Q99 FACT-BASED Constants

Which of the following correctly represents a string constant in C?

A. 'Hello' **B.** "Hello"

C. Hello **D.** (Hello)

Ans: B

Explanation:
A string constant in C is a sequence of characters enclosed within double quotes, such as "Hello".

Q100 COMPARISON-BASED Constants

What is the main difference between a character constant and a string constant in C?

A. Character constants use double quotes; string constants use single quotes

B. Character constants hold exactly one character in single quotes; string constants hold a sequence of characters in double quotes

C. Both are identical in memory representation

D. String constants cannot contain letters

Ans: B

Explanation:
A character constant represents a single character within single quotes, while a string constant represents a sequence of characters (including none) within double quotes and is terminated internally with a null character.

Q101 CONCEPTUAL Constants

What is a 'symbolic constant' in C?

A. A constant that changes its value during execution

B. A name given to a constant value using the #define preprocessor directive

C. Any variable declared with 'const' keyword only

D. A built-in mathematical constant like pi predefined in C

Ans: B

Explanation:
A symbolic constant is created by associating a name with a fixed value, typically using #define, allowing that name to be used throughout the program in place of the literal value.

Q102 APPLICATION-ORIENTED Constants

Which of the following correctly creates a symbolic constant for the value of PI?

A. int PI = 3.14;

B. #define PI 3.14

C. const PI = 3.14;

D. PI = 3.14;

Ans: B

Explanation:
#define PI 3.14 is the correct preprocessor directive syntax to create a symbolic constant named PI representing the value 3.14.

Q103 ADVANTAGES Constants

What is one major advantage of using symbolic constants in a C program?

A. They slow down compilation significantly

B. They make the program easier to read, modify, and maintain by allowing a single point of change

C. They are mandatory for every program to compile

D. They prevent the use of any numeric values

Ans: B

Explanation:
Symbolic constants improve code readability and maintainability since changing the value in one #define statement automatically updates it everywhere it is used.

Q104 FACT-BASED Constants

Which of the following is an example of an integer constant written in octal notation in C?

A. 0x1A

B. 017

C. 1.7

D. '7'

Ans: B

Explanation:
An integer constant prefixed with a leading 0 (like 017) is interpreted by the C compiler as an octal (base-8) number.

Q105 FACT-BASED Constants

Which prefix is used in C to represent a hexadecimal integer constant?

A. 0

B. 0x or 0X

C. #

D. h

Ans: B

Explanation:
Hexadecimal integer constants in C are written with a leading 0x or 0X, such as 0x1A, representing base-16 numbers.

Q106 OUTPUT-BASED Constants

What type of constant is represented by the literal 1.5e3 in C?

A. Integer constant

B. Floating constant in exponential (scientific) notation

C. Character constant

D. String constant

Ans: B

Explanation:
1.5e3 is a floating constant expressed in exponential notation, equivalent to 1.5 × 10^3, i.e., 1500.0.

Q107 CONCEPTUAL Constants

Why must character constants in C be enclosed in single quotes rather than double quotes?

A. Single quotes have no special meaning

B. Single quotes specifically denote a single character constant, while double quotes denote a string

C. It is purely a stylistic preference with no compiler distinction

D. Double quotes are reserved only for comments

Ans: B

Explanation:
C syntax differentiates single quotes for character constants and double quotes for string constants; using the wrong type would change the meaning or cause an error.

Q108 TRICKY Constants

What is the actual stored length (in characters, including terminator) of the string constant "Hi"?

A. 2 characters only

B. 3 characters: 'H', 'i', and a null terminator '\0'

C. 1 character

D. 4 characters

Ans: B

Explanation:
C automatically appends a null character ('\0') at the end of every string constant, so "Hi" actually occupies 3 bytes in memory.

Q109 IDENTIFICATION Constants

Identify the type of constant: -45

A. Floating constant

B. Integer constant (negative)

C. Character constant

D. Symbolic constant

Ans: B

Explanation:
-45 is a whole number with a negative sign and no decimal point, making it a valid (signed) integer constant.

Q110 IDENTIFICATION Constants

Identify the type of constant: '9'

A. Integer constant with value 9

B. Character constant representing the character '9' (its ASCII value, not numeric 9)

C. Floating constant

D. String constant

Ans: B

Explanation:
'9' enclosed in single quotes is a character constant representing the character symbol '9', stored using its ASCII code, not the integer value 9.

Q111 STATEMENT-BASED Constants

Statement: 'A symbolic constant defined with #define is processed by the compiler at runtime.' Is this correct?

A. True

B. False — symbolic constants defined with #define are substituted by the preprocessor before compilation begins

C. True, only for floating constants

D. True, only in ANSI C

Ans: B

Explanation:
#define is a preprocessor directive; the substitution of the symbolic constant with its value occurs during the preprocessing stage, before actual compilation.

Q112 FACT-BASED Constants

Which of the following is a valid floating constant in C?

A. 5
B. 5.0
C. '5'
D. "5"

Ans: B

Explanation:
5.0 contains a decimal point and represents a real number, making it a valid floating constant.

Q113 PREVIOUS-YEAR PATTERN Constants

Which of the following best describes all valid constant types specified for the OAVS syllabus (int, float, double, char related)?

A. Integer constants, Floating constants, Character constants, String constants, Symbolic constants
B. Only Boolean constants
C. Only Array constants
D. Pointer constants and Union constants

Ans: A

Explanation:
As per the syllabus, the constants covered are integer, floating, character, string, and symbolic constants — all directly related to int, float, double, and char usage.

Q114 APPLICATION-ORIENTED Constants

A programmer wants to fix the value of MAX_MARKS as 100 throughout a program without it ever changing. Which is the most appropriate method using symbolic constants?

A. int MAX_MARKS; MAX_MARKS = 100;
B. #define MAX_MARKS 100
C. MAX_MARKS == 100;
D. float MAX_MARKS = 100;

Ans: B

Explanation:
#define MAX_MARKS 100 creates a symbolic constant that the preprocessor replaces throughout the code, ensuring the value 100 is fixed and consistent wherever MAX_MARKS is used.

Q115 TRICKY Constants

Which of the following is NOT a valid integer constant in standard C?

A. 123
B. -123
C. 12.3
D. 0

Ans: C

Explanation:
12.3 contains a decimal point, which makes it a floating constant, not an integer constant.

Q116 CONCEPTUAL Constants

Can a string constant in C contain zero characters (an empty string)?

A. No, a string must have at least one character
B. Yes, "" is a valid empty string constant containing only the null terminator
C. No, this would cause a compilation error
D. Only character constants can be empty

Ans: B

Explanation:
"" is a valid string constant representing an empty string; internally it consists solely of the null character '\0'.

Q117 FILL-IN-THE-BLANK Constants

A constant value defined using the #define directive and given a name is called a _____ constant.

A. Floating
B. Character
C. Symbolic
D. Integer

Ans: C

Explanation:
A constant created by naming a fixed value via #define is specifically termed a symbolic constant.

Q118 COMPARISON-BASED Constants

How does an integer constant differ from a floating constant in terms of representation?

A. Integer constants always require a decimal point; floating constants never do

B. Integer constants have no decimal point and represent whole numbers; floating constants include a decimal point or exponent to represent real numbers

C. Both must be enclosed in quotes

D. There is no difference

Ans: B

Explanation:
Integer constants are whole numbers without a fractional part, whereas floating constants explicitly include a decimal point or an exponential notation to represent real numbers.

Q119 ASSERTION-REASON Constants

Assertion (A): "5" and '5' represent the same value and type in C. Reason (R): Both use quotation marks. Choose the correct option.

A. Both A and R are true and R explains A

B. A is false; "5" is a string constant while '5' is a character constant — they differ in type despite both using quotes

C. A is true but R is false

D. Both A and R are false but unrelated to quotes

Ans: B

Explanation:
"5" is a string constant (an array of characters ending in null), while '5' is a character constant representing a single ASCII value; they are different in both type and internal representation.

Q120 APPLICATION-ORIENTED Constants

Which constant type would be most suitable to store the grade letter 'A' for a student in a simple program?

A. Integer constant

B. Character constant: 'A'

C. Floating constant

D. Symbolic constant only

Ans: B

Explanation:
Since a single grade letter is being stored, a character constant such as 'A' (in single quotes) is the most appropriate representation.

Chapter 5: Type Qualifiers

Q121 DEFINITION-BASED Type Qualifiers

What is the purpose of a 'type qualifier' in C, such as long, short, signed, or unsigned?

A. To completely change the basic data type into a new category

B. To modify the storage size and/or range/sign behavior of a basic data type

C. To define a new keyword

D. To create comments in the program

Ans: B

Explanation:
Type qualifiers like long, short, signed, and unsigned modify the storage size or the sign interpretation of a basic data type such as int or char, without creating a fundamentally new type category.

Q122 BASIC Type Qualifiers

Which of the following is a valid type-qualified declaration in C?

A. long int marks;

B. longint marks;

C. int long marks marks;

D. long-int marks;

Ans: A

Explanation:
long int marks; is correctly written with the qualifier 'long' preceding the base type 'int', forming a valid declaration.

Q123 CONCEPTUAL Type Qualifiers

What does the 'unsigned' qualifier do when applied to an int variable?

A. It allows the variable to store both positive and negative numbers as usual

B. It restricts the variable to store only non-negative values, effectively doubling the positive range

C. It converts the int into a float

D. It has no actual effect

Ans: B

Explanation:
The unsigned qualifier removes the sign bit's role in representing negative numbers, allowing the full bit range to represent only non-negative (zero and positive) values, effectively extending the maximum positive value.

Q124 CONCEPTUAL Type Qualifiers

What is the default sign behavior of an 'int' in C if no qualifier is explicitly specified?

A. It is always unsigned by default

B. It is signed by default, capable of representing both positive and negative values

C. It depends randomly on each program run

D. It cannot store any value without an explicit qualifier

Ans: B

Explanation:
Unless explicitly declared otherwise, an int in C is signed by default, meaning it can represent both negative and positive integers.

Q125 COMPARISON-BASED Type Qualifiers

What is the main difference between 'short int' and 'long int'?

A. short int generally uses less memory and has a smaller range than long int, which uses more memory and has a larger range

B. They are exactly identical in every compiler

C. short int can only store characters

D. long int cannot store negative numbers

Ans: A

Explanation:
short int typically occupies less memory (commonly 2 bytes) with a smaller range, while long int typically occupies more memory (commonly 4 or 8 bytes) supporting a larger range of values.

Q126 FACT-BASED Type Qualifiers

Which of the following qualifiers, when combined with int, is used WITHOUT changing the word 'int' explicitly (i.e., can be used alone)?

A. long (can be used alone, implying long int)

B. double (cannot be used as a qualifier)

C. char (is a base type, not a qualifier)

D. void (is a base type, not a qualifier)

Ans: A

Explanation:
In C, 'long' can be used alone as a shorthand for 'long int', since the qualifier implicitly assumes the base type int when omitted.

Q127 USES Type Qualifiers

In which scenario would the 'unsigned' qualifier be most appropriately used?

A. When storing a temperature that can go below zero

B. When storing a quantity that is always non-negative, such as a count of items or an age

C. When storing profit/loss values that can be negative

D. When storing pi (3.14159)

Ans: B

Explanation:
Since 'unsigned' only represents non-negative values, it is best suited for quantities that logically never go below zero, such as counts, ages, or array indices.

Q128 TRICKY Type Qualifiers

What is a likely consequence of assigning a negative value to an unsigned int variable in C?

A. It causes a compilation error every time

B. The negative value gets converted/wrapped into a very large positive number due to how unsigned representation works

C. It is automatically stored as zero

D. It crashes the program immediately

Ans: B

Explanation:
Assigning a negative value to an unsigned variable causes it to wrap around using modular arithmetic, resulting in a very large positive number instead of an error.

Q129 FACT-BASED Type Qualifiers

Which type qualifier, when applied to 'char', allows it to explicitly represent negative values as well as positive ones?

A. unsigned char

B. signed char

C. long char

D. short char

Ans: B

Explanation:
signed char explicitly ensures the char variable can represent both negative and positive values, since plain 'char' sign behavior can be implementation-defined.

Q130 CONCEPTUAL Type Qualifiers

Why might the behavior of plain 'char' (without signed/unsigned) be considered ambiguous across different compilers?

A. Because char never stores any value

B. Because ANSI C leaves it implementation-defined whether plain char is treated as signed or unsigned

C. Because char is identical to int in every compiler

D. Because char can only be used with arrays

Ans: B

Explanation:
The C standard does not mandate whether a plain 'char' is signed or unsigned by default; this is left to the implementation, which is why signed char or unsigned char is sometimes used explicitly for clarity.

Q131 COMPARISON-BASED Type Qualifiers

How do 'signed' and 'unsigned' qualifiers differ in their effect on the same number of bits?

A. signed splits the range between negative and positive values; unsigned uses the full range for non-negative values only

B. unsigned always uses more bits than signed

C. Both represent identical ranges in all cases

D. signed cannot represent zero

Ans: A

Explanation:
For the same bit width, signed types split the representable range to cover both negative and positive numbers, whereas unsigned types use the entire range exclusively for zero and positive values.

Q132 APPLICATION-ORIENTED Type Qualifiers

A program needs to store very large positive whole numbers beyond the normal int range, but never negative numbers. Which combination of qualifiers would be most suitable?

A. short int

B. signed char

C. unsigned long int

D. float

Ans: C

Explanation:
unsigned long int combines extended storage size (long) with a purely non-negative range (unsigned), making it suitable for very large positive whole numbers.

Q133 FACT-BASED Type Qualifiers

Which of the following is generally the correct relative ordering of storage size (smallest to largest) among integer-related qualifiers on typical systems?

A. short int ≤ int ≤ long int

B. long int ≤ int ≤ short int

C. int ≤ short int ≤ long int

D. All are always exactly the same size

Ans: A

Explanation:
On most systems, short int requires the least memory, followed by int, with long int typically requiring equal or more memory, reflecting increasing range capacity.

Q134 CHARACTERISTICS Type Qualifiers

Which of the following is a key characteristic of the 'long' qualifier?

A. It always decreases the storage size of a variable

B. It typically increases the storage size, allowing a larger range of values to be represented

C. It converts integers into characters

D. It is used only with floating constants, never with int

Ans: B

Explanation:
The 'long' qualifier typically increases the number of bytes allocated to a variable, thereby extending the range of values it can represent.

Q135 STATEMENT-BASED Type Qualifiers

Statement: 'unsigned' and 'signed' can only be applied to int, never to char.' Is this statement correct?

A. True

B. False — both unsigned and signed can also be applied to char (e.g., signed char, unsigned char)

C. True, only in ANSI C90

D. True, but only for double

Ans: B

Explanation:
Both 'signed' and 'unsigned' qualifiers can be validly applied to char as well as int, producing types like signed char and unsigned char.

Q136 FILL-IN-THE-BLANK Type Qualifiers

The qualifier _____ is used to reduce the storage size and range of the basic int type.

A. long
B. short
C. unsigned
D. signed

Ans: B

Explanation:
The 'short' qualifier reduces the typical storage allocation for an int, resulting in a smaller representable range compared to a plain int.

Q137 PREVIOUS-YEAR PATTERN Type Qualifiers

Which of the following correctly lists all FOUR type qualifiers specified in the OAVS syllabus?

A. long, short, signed, unsigned
B. static, extern, auto, register
C. const, volatile, restrict, inline
D. public, private, protected, friend

Ans: A

Explanation:
As per the syllabus, the four type qualifiers covered are long, short, signed, and unsigned, which modify the size and sign behavior of basic data types.

Q138 TRICKY Type Qualifiers

Can 'long' and 'unsigned' be combined together in a single variable declaration?

A. No, only one qualifier is allowed per variable
B. Yes, e.g., 'unsigned long int' is a valid combined declaration
C. No, this combination always causes an error
D. Yes, but only with float

Ans: B

Explanation:
Multiple qualifiers can be combined, such as in 'unsigned long int', to simultaneously extend storage size and restrict the range to non-negative values.

Q139 CONCEPTUAL Type Qualifiers

What is the primary purpose of qualifiers in optimizing memory usage in a C program?

A. They have no relation to memory usage

B. Using 'short' where a smaller range suffices can save memory compared to using a plain or long int

C. They always increase memory usage regardless of choice

D. Memory usage is unrelated to data type qualifiers

Ans: B

Explanation:
Choosing an appropriately sized qualifier, like 'short' for small-range data, helps in optimizing and reducing the overall memory footprint of a program.

Q140 OUTPUT-BASED Type Qualifiers

Which of the following is the correct way to declare an unsigned short integer variable named 'count'?

A. unsigned short int count;

B. short unsigned count count;

C. count unsigned short;

D. int unsigned short count = ;

Ans: A

Explanation:
unsigned short int count; correctly places the qualifiers 'unsigned' and 'short' before the base type 'int' and the identifier 'count'.

Q141 USES Type Qualifiers

Why might a programmer choose 'long double' over 'double' in a calculation (introductory concept)?

A. long double always uses less memory than double

B. long double offers extended precision and range beyond standard double, useful for highly precise calculations

C. long double is identical to int

D. There is no such qualifier combination in C

Ans: B

Explanation:
long double extends the precision and range beyond what a standard double offers, making it useful in calculations requiring extra accuracy.

Q142 COMPARISON-BASED Type Qualifiers

Which statement best compares 'signed int' and plain 'int' in standard C?

A. They are functionally identical, since int is signed by default

B. signed int can store negative numbers while plain int cannot

C. Plain int is always unsigned

D. signed int uses double the memory of plain int

Ans: A

Explanation:
Since int is signed by default in C, explicitly writing 'signed int' produces functionally the same behavior as simply writing 'int'.

Q143 APPLICATION-ORIENTED Type Qualifiers

A banking application must track account balances that could be negative (overdraft) and require precision. Which is the most semantically appropriate basic qualifier-related choice for the integer part of computations needing a larger range?

A. unsigned short int

B. signed long int

C. unsigned char

D. short unsigned int

Ans: B

Explanation:
Since balances may be negative and could require a larger range, 'signed long int' is most appropriate, supporting both negative values and extended range.

Q144 ASSERTION-REASON Type Qualifiers

Assertion (A): unsigned int has a larger maximum positive value than a signed int of the same size. Reason (R): unsigned int dedicates all bits to representing magnitude with no sign bit needed for negative numbers. Choose the correct option.

A. Both A and R are true and R correctly explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: A

Explanation:
Both statements are true and related: since unsigned types use all available bits for magnitude (no sign bit needed), they can represent a higher maximum positive value than the equivalent signed type.

Q145 FACT-BASED Type Qualifiers

Which of the following is generally an INVALID qualifier-base type combination in standard ANSI C?

A. long int

B. short int

C. long float

D. unsigned int

Ans: C

Explanation:
long float is not a standard valid combination in ANSI C; the correct way to get extended floating precision is to use 'double' or 'long double', not 'long float'.

Chapter 6: Escape Sequences

Q146 DEFINITION-BASED — Escape Sequences

What is an 'escape sequence' in C?

A. A keyword used to exit a loop

B. A combination of a backslash followed by a character, used to represent special, non-printable, or control characters

C. A type of arithmetic operator

D. A symbolic constant defined with #define

Ans: B

Explanation:
An escape sequence is a special character combination starting with a backslash (\) that represents characters which cannot be typed directly or have special meaning, such as newline or tab.

Q147 FACT-BASED — Escape Sequences

What does the escape sequence \n represent in C?

A. Tab space

B. Newline (moves cursor to the next line)

C. Backspace

D. Null character

Ans: B

Explanation:
\n represents the newline character, which moves the cursor to the beginning of the next line when printed.

Q148 FACT-BASED — Escape Sequences

What does the escape sequence \b represent in C?

A. Bell sound

B. Backspace (moves cursor one position back)

C. Backslash character

D. Beginning of line

Ans: B

Explanation:
\b represents the backspace character, which moves the cursor one position backward, effectively erasing the previous character in display contexts.

Q149 FACT-BASED Escape Sequences

What does the escape sequence \t represent in C?

A. Horizontal tab

B. Top of file marker

C. True boolean value

D. Time stamp

Ans: A

Explanation:
\t represents a horizontal tab, which inserts a tab space, commonly used for aligning output.

Q150 FACT-BASED Escape Sequences

What does the escape sequence \r represent in C?

A. Reverse text

B. Carriage return (moves cursor to beginning of the current line)

C. Remove character

D. Run program

Ans: B

Explanation:
\r represents a carriage return, which moves the cursor to the beginning of the current line without advancing to a new line.

Q151 FACT-BASED Escape Sequences

Which escape sequence is used in C to print a single backslash character (\) itself?

A. \\

B. \b

C. \n

D. \"

Ans: A

Explanation:
\\ (double backslash) is the escape sequence used to print a literal single backslash character, since a single backslash alone would be interpreted as the start of another escape sequence.

Q152 FACT-BASED Escape Sequences

Which escape sequence is used to print a single quote character (') within a character constant?

A. \'
B. \"
C. \n
D. \0

Ans: A

Explanation:
\' is the escape sequence used to represent a single quote character, which is necessary because an unescaped single quote would otherwise terminate a character constant.

Q153 FACT-BASED Escape Sequences

Which escape sequence is used to print a double quote character (") within a string?

A. \'
B. \"
C. \\
D. \a

Ans: B

Explanation:
\" is the escape sequence used to insert a literal double quote character inside a string constant, since an unescaped double quote would otherwise terminate the string.

Q154 FACT-BASED Escape Sequences

Which escape sequence represents the 'null character' in C, commonly used to terminate strings?

A. \0
B. \n
C. \t
D. \a

Ans: A

Explanation:
\0 represents the null character, which has an ASCII value of 0 and is automatically used by C to mark the end of string constants.

Q155 FACT-BASED Escape Sequences

Which escape sequence in standard ANSI C produces an audible alert (bell sound) on supporting systems?

A. \a
B. \v
C. \f
D. \b

Ans: A

Explanation:
\a is the 'alert' or 'bell' escape sequence, which triggers an audible or visual alert signal on systems that support it.

Q156 FACT-BASED Escape Sequences

What does the escape sequence \f represent in C?

A. Form feed (used to move to a new page in printed output)
B. False boolean value
C. File pointer
D. Function call marker

Ans: A

Explanation:
\f represents a form feed character, traditionally used to advance the page on printers, though its visible effect on screens may vary.

Q157 FACT-BASED Escape Sequences

What does the escape sequence \v represent in C?

A. Vertical tab
B. Variable declaration
C. Void return
D. Value separator

Ans: A

Explanation:
\v represents a vertical tab character, which moves the cursor down to the next vertical tab stop.

Q158 OUTPUT-BASED Escape Sequences

What will be the output behavior of printf("Hello\nWorld");?

A. HelloWorld printed on a single line

B. Hello and World printed on two separate lines

C. An error will occur

D. Only 'Hello' will be printed

Ans: B

Explanation:
The \n escape sequence causes a line break, so 'Hello' is printed first, the cursor moves to a new line, and then 'World' is printed.

Q159 OUTPUT-BASED Escape Sequences

What will printf("A\tB"); display, considering the effect of the escape sequence?

A. A and B printed with a tab space between them

B. AB printed together with no space

C. Only A is printed

D. An error occurs due to invalid syntax

Ans: A

Explanation:
The \t escape sequence inserts a horizontal tab, so the output displays 'A' followed by a tab space and then 'B'.

Q160 CONCEPTUAL Escape Sequences

Why are escape sequences necessary in C, given that the C character set doesn't directly provide control characters as typeable symbols?

A. They are not necessary; everything can be typed directly

B. They allow representation of non-printable or special-purpose characters (like newline, tab) that cannot be typed directly within strings

C. They are purely decorative with no functional use

D. They are only used for mathematical operations

Ans: B

Explanation:
Escape sequences provide a way to represent control characters such as newline, tab, or backspace that have no direct typeable symbol but are essential for formatting output.

Q161 COMPARISON-BASED Escape Sequences

How does \n differ from \r in terms of cursor behavior?

A. \n moves to a new line; \r moves the cursor to the beginning of the current line without creating a new line

B. Both behave exactly the same way

C. \r always deletes the current line; \n does nothing

D. \n is used only in printf, \r only in scanf

Ans: A

Explanation:
\n (newline) advances the cursor to the start of the next line, while \r (carriage return) returns the cursor to the beginning of the current line without moving down.

Q162 APPLICATION-ORIENTED Escape Sequences

A programmer wants to display the text: She said, "Good morning!" using printf. Which escape sequence must be used around 'Good morning!'?

A. \'

B. \"

C. \\

D. \n

Ans: B

Explanation:
Since double quotes are needed within a string literal, the \" escape sequence must be used to embed literal double quote characters without ending the string prematurely.

Q163 TRICKY Escape Sequences

What would happen if a programmer mistakenly wrote "C:\new\test" as a file path string, without proper escaping?

A. It would work exactly as intended with no issues

B. \n inside \new would be misinterpreted as a newline escape sequence, corrupting the intended path

C. It would cause a compilation error always

D. Backslashes are ignored entirely in strings

Ans: B

Explanation:
Since \n is a valid escape sequence, the substring \new would have its \n interpreted as a newline character, not as a literal backslash followed by 'n', corrupting the intended file path.

Q164 FILL-IN-THE-BLANK Escape Sequences

To print a literal backslash character followed by the letter n (i.e., the two characters \ and n, not a newline), a programmer should write _____ in the string.

A. \n
B. \\n
C. \\\\n
D. \bn

Ans: B

Explanation:
Writing \\n produces a literal backslash (via \\) followed by the letter n, rather than triggering the newline escape sequence.

Q165 PREVIOUS-YEAR PATTERN Escape Sequences

Which of the following correctly matches the escape sequence with its standard meaning: \t, \b, \r, \\?

A. Tab, Backspace, Carriage return, Backslash
B. Tab, Bell, Return, Backward
C. Time, Break, Run, Back
D. Type, Byte, Return, Bracket

Ans: A

Explanation:
\t represents tab, \b represents backspace, \r represents carriage return, and \\ represents a literal backslash — matching the standard ANSI C escape sequence definitions.

Q166 STATEMENT-BASED Escape Sequences

Statement: 'Every escape sequence in C begins with a backslash (\) character.' Is this statement correct?

A. True
B. False
C. True, except for \n
D. True, except for \t

Ans: A

Explanation:
All standard escape sequences in C are formed by a backslash followed by a specific character, making this statement universally true for escape sequences.

Q167 CONCEPTUAL Escape Sequences

In what sense are escape sequences considered 'single characters' even though they are written using two symbols?

A. They are not single characters; they always occupy two bytes

B. Although written with two symbols (backslash + character), the compiler interprets and stores them as a single character internally

C. They are treated as full strings, not characters

D. They only work inside comments

Ans: B

Explanation:
Despite their two-symbol textual representation, escape sequences are interpreted by the compiler as representing a single character value, such as the newline character.

Q168 APPLICATION-ORIENTED Escape Sequences

Which escape sequence would be most useful for creating a simple text-based 'typing correction' effect (deleting the last printed character) in console output?

A. \n

B. \b

C. \t

D. \f

Ans: B

Explanation:
\b (backspace) moves the cursor back one position, which can visually simulate the deletion of the previously printed character in a console.

Q169 FACT-BASED Escape Sequences

Which of the following is the correct escape sequence to represent the ASCII null character used to terminate strings in C?

A. \null

B. \0

C. \z

D. \e

Ans: B

Explanation:
\0 is the standard escape sequence representing the null character (ASCII value 0), which C automatically appends to terminate string constants.

Q170 TRICKY

Escape Sequences

Why can't a programmer simply press the Tab key inside a string literal to insert a tab space without using \t, in many cases?

A. Pressing Tab inside a literal works exactly like \t with no difference and is always recommended

B. While it may sometimes work depending on the editor, using \t is the standard, portable way to guarantee the tab character is correctly embedded across compilers/editors

C. It is strictly forbidden by the C standard to use Tab anywhere

D. Tab key presses are converted into newlines automatically

Ans: B

Explanation:
Although raw tab characters might be accepted by some editors, \t is the standard, portable way to explicitly and reliably represent a tab character within string literals across different systems.

Q171 COMPARISON-BASED

Escape Sequences

How do \a and \f differ in their typical usage and effect?

A. \a produces an alert/bell sound; \f produces a form feed (page break in print contexts) — they serve very different purposes

B. Both produce identical newline effects

C. \a is used only in comments; \f only in keywords

D. There is no difference; they are interchangeable

Ans: A

Explanation:
\a (alert) and \f (form feed) serve distinctly different purposes — one for an audible/visual alert and the other historically for advancing to a new page in printed output.

Chapter 7: Arithmetic Expressions and Built-in Operators

Q172 DEFINITION-BASED — Arithmetic Expressions & Operators

What is an 'arithmetic expression' in C?

A. A statement that only declares variables

B. A combination of operands and arithmetic operators that evaluates to a numeric value

C. A type of comment

D. A preprocessor directive

Ans: B

Explanation:
An arithmetic expression combines variables, constants, and arithmetic operators (like +, -, *, /, %) to compute a resulting numeric value.

Q173 BASIC — Arithmetic Expressions & Operators

Which of the following are the basic arithmetic operators available in C?

A. +, -, *, /, %

B. &&, ||, !

C. ==, !=, <, >

D. &, |, ^, ~

Ans: A

Explanation:
The basic arithmetic operators in C are addition (+), subtraction (-), multiplication (*), division (/), and modulus (%).

Q174 FACT-BASED — Arithmetic Expressions & Operators

What does the modulus operator (%) do in C?

A. Performs floating-point division

B. Returns the remainder of an integer division

C. Multiplies two numbers

D. Compares two values for equality

Ans: B

Explanation:
The modulus operator (%) returns the remainder when one integer is divided by another, and is only valid for integer operands in standard C.

Q175 OUTPUT-BASED Arithmetic Expressions & Operators

What is the result of the expression 17 % 5 in C?

A. 3.4 **B.** 2

C. 3 **D.** 12

Ans: B

Explanation:
17 divided by 5 gives a quotient of 3 with a remainder of 2; the modulus operator returns this remainder, which is 2.

Q176 OUTPUT-BASED Arithmetic Expressions & Operators

What is the result of the expression 9 / 2 when both operands are of type int in C?

A. 4.5 **B.** 4

C. 5 **D.** Error

Ans: B

Explanation:
Since both operands are integers, C performs integer division, discarding the fractional part, so 9/2 evaluates to 4.

Q177 OUTPUT-BASED Arithmetic Expressions & Operators

What is the result of the expression 9.0 / 2 in C?

A. 4 **B.** 4.5

C. 5 **D.** Error

Ans: B

Explanation:
Since one operand (9.0) is a floating-point value, C performs floating-point division, yielding the precise result 4.5.

Q178 CONCEPTUAL Arithmetic Expressions & Operators

What is meant by 'precedence' of operators in an arithmetic expression?

A. The order in which variables are declared

B. The priority that determines which operator is evaluated first when multiple operators appear in an expression

C. The size of the data type used

D. The number of operands in an expression

Ans: B

Explanation:
Operator precedence determines the order in which operators are evaluated when an expression contains more than one operator, with higher-precedence operators evaluated first.

Q179 CONCEPTUAL Arithmetic Expressions & Operators

What is meant by 'associativity' of operators?

A. The total number of operators used

B. The direction (left-to-right or right-to-left) in which operators of the same precedence level are evaluated

C. Whether an operator can be used with strings

D. The memory address of an operand

Ans: B

Explanation:
Associativity defines the evaluation direction (left-to-right or right-to-left) for operators sharing the same precedence level when they appear consecutively in an expression.

Q180 PRECEDENCE Arithmetic Expressions & Operators

In the expression 5 + 3 * 2, which operation is performed first according to operator precedence?

A. Addition (5 + 3) first

B. Multiplication (3 * 2) first

C. Both simultaneously

D. Neither; it causes an error

Ans: B

Explanation:
Multiplication has higher precedence than addition, so 3 * 2 is evaluated first, giving 6, and then 5 + 6 is computed, resulting in 11.

Q181 OUTPUT-BASED Arithmetic Expressions & Operators

What is the value of the expression 5 + 3 * 2 in C?

A. 16
B. 11
C. 13
D. 10

Ans: B

Explanation:
Following operator precedence, 3 * 2 = 6 is computed first, then 5 + 6 = 11, giving the final result of 11.

Q182 OUTPUT-BASED Arithmetic Expressions & Operators

What is the value of the expression (5 + 3) * 2 in C?

A. 11
B. 16
C. 13
D. 10

Ans: B

Explanation:
Parentheses have the highest precedence, so 5 + 3 = 8 is computed first, and then 8 * 2 = 16 is the final result.

Q183 ASSOCIATIVITY Arithmetic Expressions & Operators

What is the associativity of the multiplication (*) and division (/) operators when they appear together at the same precedence level?

A. Right to left
B. Left to right
C. They cannot appear together
D. Associativity does not apply to these operators

Ans: B

Explanation:
Multiplication and division share the same precedence and are evaluated with left-to-right associativity when they appear together in an expression.

Q184 OUTPUT-BASED Arithmetic Expressions & Operators

What is the result of the expression 20 / 4 * 2 in C, considering left-to-right associativity?

A. 2.5
B. 10
C. 40
D. 5

Ans: B

Explanation:
With left-to-right associativity, 20 / 4 = 5 is computed first, then 5 * 2 = 10, giving the final result of 10.

Q185 DEFINITION-BASED Arithmetic Expressions & Operators

What is the difference between a 'unary' and a 'binary' operator?

A. A unary operator requires one operand; a binary operator requires two operands
B. A unary operator requires two operands; binary requires one
C. Both require exactly three operands
D. There is no difference

Ans: A

Explanation:
A unary operator acts on a single operand (e.g., unary minus: -x), while a binary operator acts on two operands (e.g., a + b).

Q186 FACT-BASED Arithmetic Expressions & Operators

Which of the following is an example of a unary operator in C?

A. + (addition)
B. - (unary minus, as in -x)
C. * (multiplication)
D. % (modulus)

Ans: B

Explanation:
The unary minus operator, applied as -x, negates a single operand, making it a unary operator, unlike the binary versions of +, *, and %.

Q187 FACT-BASED Arithmetic Expressions & Operators

Which of the following operators are classified as increment/decrement unary operators in C?

A. ++ and --

B. + and -

C. * and /

D. && and ||

Ans: A

Explanation:
The increment (++) and decrement (--) operators are unary operators that increase or decrease the value of a single operand by 1.

Q188 COMPARISON-BASED Arithmetic Expressions & Operators

What is the key difference between pre-increment (++x) and post-increment (x++) in terms of expression evaluation?

A. Pre-increment increases the value after using it in the expression; post-increment increases it before

B. Pre-increment increases the value first and then uses the updated value in the expression; post-increment uses the current value first, then increases it

C. Both behave identically in every context

D. Post-increment cannot be used in expressions

Ans: B

Explanation:
Pre-increment (++x) increments the variable's value before it is used in the surrounding expression, while post-increment (x++) uses the current value first and increments afterward.

Q189 OUTPUT-BASED Arithmetic Expressions & Operators

If int x = 5; int y = ++x; what will be the value of y after execution?

A. 5

B. 6

C. 4

D. Undefined

Ans: B

Explanation:
Pre-increment (++x) increases x to 6 before assigning its value to y, so y receives the updated value 6.

Q190 OUTPUT-BASED

Arithmetic Expressions & Operators

If int x = 5; int y = x++; what will be the value of y after execution?

A. 6

B. 5

C. 4

D. Undefined

Ans: B

Explanation:

Post-increment (x++) uses the current value of x (5) for the assignment to y first, and only afterward increments x to 6.

Q191 SIMPLE EXPRESSION ANALYSIS

Arithmetic Expressions & Operators

Evaluate the expression: 10 - 2 + 3 in standard left-to-right associativity for + and -.

A. 5

B. 11

C. 15

D. 9

Ans: B

Explanation:

Since + and - share the same precedence and associate left to right, 10 - 2 = 8 is computed first, then 8 + 3 = 11.

Q192 APPLICATION-ORIENTED

Arithmetic Expressions & Operators

A programmer writes the expression: 7 + 3 * 2 - 4 / 2. What is the evaluated result?

A. 9

B. 11

C. 7

D. 13

Ans: B

Explanation:

Following precedence, 3*2=6 and 4/2=2 are computed first, giving 7 + 6 - 2; evaluating left to right, 7+6=13, then 13-2=11, so the final result is 11.

Q193 TRICKY

Arithmetic Expressions & Operators

Why is parentheses usage often recommended in complex arithmetic expressions even when precedence rules would give the correct result anyway?

A. Parentheses are mandatory by C syntax in every expression

B. Parentheses improve readability and reduce the chance of programmer error in interpreting the intended order of evaluation

C. Parentheses change the data type of the result

D. Parentheses always slow down execution significantly

Ans: B

Explanation:
While precedence rules ensure a defined evaluation order, using parentheses makes the programmer's intent explicit and reduces the risk of misreading or miscalculating complex expressions.

Q194 BUILT-IN OPERATORS

Arithmetic Expressions & Operators

Among the C built-in operators related to arithmetic, which one specifically requires both operands to be of integer type for valid use?

A. + (addition)

B. % (modulus)

C. - (subtraction)

D. * (multiplication)

Ans: B

Explanation:
The modulus operator (%) in standard C operates only on integer operands; using it with floating-point operands directly is not valid in C.

Q195 CONCEPTUAL

Arithmetic Expressions & Operators

What does it mean when we say an arithmetic operator in C is 'binary' in nature?

A. It can only operate on binary numbers (0s and 1s)

B. It requires exactly two operands to perform its operation

C. It always returns a Boolean result

D. It is used only in binary search algorithms

Ans: B

Explanation:
A binary operator, in the context of arity, simply means it requires two operands to function, such as a + b or a * b, regardless of the numeral system involved.

Q196 STATEMENT-BASED Arithmetic Expressions & Operators

Statement: 'In C, the expression a / b always returns a floating-point result regardless of the data types of a and b.' Is this true?

A. True

B. False — if both a and b are integers, integer division occurs, truncating any decimal part

C. True, only for the % operator

D. True, only when using #define

Ans: B

Explanation:
The result type of division depends on the operand types; if both are integers, C performs integer division and truncates the decimal portion rather than always producing a float.

Q197 FILL-IN-THE-BLANK Arithmetic Expressions & Operators

The operator used to find the remainder after division of two integers in C is called the _____ operator.

A. Division

B. Modulus

C. Multiplication

D. Exponent

Ans: B

Explanation:
The modulus operator (%) is specifically used to compute the remainder after integer division in C.

Q198 PREVIOUS-YEAR PATTERN Arithmetic Expressions & Operators

As per standard C precedence rules, which group of operators is evaluated before addition and subtraction in a typical arithmetic expression?

A. Multiplication, Division, and Modulus

B. Assignment operators

C. Relational operators

D. Logical operators

Ans: A

Explanation:
Multiplication (*), Division (/), and Modulus (%) have higher precedence than addition (+) and subtraction (-), so they are evaluated first in mixed expressions.

Q199 ASSERTION-REASON Arithmetic Expressions & Operators

Assertion (A): The expression 6 + 4 % 4 evaluates to 6. Reason (R): The modulus operator has higher precedence than addition. Choose the correct option.

A. Both A and R are true and R correctly explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: A

Explanation:
4 % 4 = 0 is computed first due to higher precedence of modulus over addition, then 6 + 0 = 6, confirming both the assertion and the correct reasoning behind it.

Q200 TRICKY Arithmetic Expressions & Operators

What is the result of the expression -5 % 2 in standard C (sign behavior of modulus with negative operand)?

A. 1

B. -1

C. Always 0

D. Undefined in all compilers

Ans: B

Explanation:
In C, the sign of the result of the modulus operator with a negative operand typically follows the sign of the dividend (in ANSI C99 and later, the result is implementation-defined in C89 but typically -1 for -5 % 2 on most modern systems).

Q201 APPLICATION-ORIENTED Arithmetic Expressions & Operators

A program calculates the average of three integer marks using: avg = (m1 + m2 + m3) / 3; where all variables are int. What potential issue might arise?

A. The result will always be a precise decimal value

B. Integer division may truncate any fractional part of the average, losing precision

C. This causes a compile-time error

D. The modulus operator is required instead

Ans: B

Explanation:
Since all variables are integers, the division (m1+m2+m3)/3 performs integer division, which truncates any decimal part, potentially giving an inaccurate average.

Q202 CONCEPTUAL

Why is operator precedence important when writing arithmetic expressions without parentheses?

A. It has no real importance in C

B. It determines the actual computed result, since different evaluation orders can produce different answers

C. It only affects how the code looks, not its result

D. Precedence only matters for logical operators, not arithmetic

Ans: B

Explanation:

Operator precedence directly determines the order of evaluation, which can significantly change the computed result of an expression if not properly understood.

Chapter 8: Preprocessor Directives and Header Files

Q203 DEFINITION-BASED

Preprocessor Directives & Header Files

What is a 'preprocessor directive' in C?

A. A runtime instruction executed by the CPU

B. A command processed by the preprocessor before actual compilation begins, usually starting with #

C. A type of loop structure

D. A keyword used only inside functions

Ans: B

Explanation:
Preprocessor directives are instructions, beginning with the # symbol, that are processed by the preprocessor before the compilation phase actually starts.

Q204 FACT-BASED

Preprocessor Directives & Header Files

Which preprocessor directive is used to include header files in a C program?

A. #define

B. #include

C. #ifdef

D. #pragma

Ans: B

Explanation:
#include is the preprocessor directive used to insert the content of a specified header file into the program before compilation.

Q205 SYNTAX

Preprocessor Directives & Header Files

Which of the following is the correct syntax to include the standard input-output header file in C?

A. #include <stdio.h>

B. include stdio.h;

C. #include "stdio"

D. import stdio.h;

Ans: A

Explanation:
#include <stdio.h> is the correct syntax, using angle brackets to include the standard input-output header file from the system's standard library directories.

Q206 CONCEPTUAL Preprocessor Directives & Header Files

What is the difference between using angle brackets (< >) and double quotes (" ") with #include?

A. There is no difference at all

B. Angle brackets tell the compiler to search standard system directories; double quotes typically search the current directory first, then system directories

C. Double quotes are used only for macros

D. Angle brackets are used only for user-defined files

Ans: B

Explanation:
Angle brackets direct the preprocessor to search standard system include directories, while double quotes typically prompt it to search the current/local directory first before falling back to system directories, commonly used for user-defined header files.

Q207 FACT-BASED Preprocessor Directives & Header Files

Which preprocessor directive is used to define a symbolic constant or macro in C?

A. #include

B. #define

C. #undef

D. #error

Ans: B

Explanation:
#define is the preprocessor directive used to create symbolic constants or macros, associating a name with a specific value or code substitution.

Q208 SYNTAX Preprocessor Directives & Header Files

Which of the following is the correct syntax for defining a symbolic constant MAX as 50 using #define?

A. #define MAX = 50;

B. #define MAX 50

C. define MAX 50;

D. #MAX define 50

Ans: B

Explanation:
#define MAX 50 is correctly written without an equals sign or semicolon, as #define directives do not follow normal C statement syntax.

Q209 CONCEPTUAL

Preprocessor Directives & Header Files

Why does a #define statement NOT require a semicolon at the end, unlike normal C statements?

A. Semicolons are forbidden everywhere in C

B. Because #define is a preprocessor directive, not a C statement, and is processed by the preprocessor through simple text substitution

C. It actually does require a semicolon

D. Because #define only works with strings

Ans: B

Explanation:
Since #define directives are handled by the preprocessor through direct text substitution rather than being compiled as statements, they do not follow normal statement syntax rules like requiring a semicolon.

Q210 PURPOSE

Preprocessor Directives & Header Files

What is the main purpose of preprocessor directives in a C program?

A. To execute the program faster at runtime

B. To perform certain text-based operations, like file inclusion and macro substitution, before actual compilation

C. To replace the need for a compiler entirely

D. To handle user input during execution

Ans: B

Explanation:
Preprocessor directives perform preliminary text processing tasks such as including files and substituting macros, all of which occur before the compiler translates the code.

Q211 USAGE

Preprocessor Directives & Header Files

In which phase of program translation are preprocessor directives like #include and #define actually processed?

A. During execution

B. During linking

C. During the preprocessing phase, before compilation

D. During loading

Ans: C

Explanation:
Preprocessor directives are handled exclusively during the preprocessing phase, which occurs before the source code is passed to the compiler for translation.

Q212 DEFINITION-BASED Preprocessor Directives & Header Files

What is a 'header file' in C?

A. A file containing only comments

B. A file containing declarations of standard library functions, macros, and constants that can be included into a program

C. A file used only to store program output

D. A type of executable file

Ans: B

Explanation:
A header file contains declarations for functions, macros, and constants (such as those in the standard library) that a program can use after including the file.

Q213 FACT-BASED Preprocessor Directives & Header Files

Which header file must typically be included to use the printf() and scanf() functions in C?

A. stdlib.h

B. stdio.h

C. math.h

D. string.h

Ans: B

Explanation:
stdio.h (Standard Input Output header) contains the declarations for printf(), scanf(), and other standard I/O functions used in C programs.

Q214 FACT-BASED Preprocessor Directives & Header Files

Which header file is commonly associated with general utility functions like malloc(), exit(), as introductory concept of standard headers?

A. stdio.h

B. stdlib.h

C. conio.h

D. ctype.h

Ans: B

Explanation:
stdlib.h (Standard Library header) provides declarations for general utility functions, including memory allocation and program termination functions.

Q215 USES

Preprocessor Directives & Header Files

What is one major advantage of using header files in C programming?

A. They make the program run without a compiler

B. They allow reuse of predefined functions and declarations without rewriting their code in every program

C. They eliminate the need for any preprocessor directives

D. They automatically fix all logical errors in the program

Ans: B

Explanation:
Header files allow programmers to reuse standard or custom function declarations and definitions across multiple programs without rewriting the underlying code each time.

Q216 INCLUSION METHODS

Preprocessor Directives & Header Files

Which of the following represents the correct method to include a user-defined header file named 'myheader.h' located in the same directory as the source file?

A. #include <myheader.h>

B. #include "myheader.h"

C. #include (myheader.h)

D. #include myheader.h;

Ans: B

Explanation:
User-defined header files in the same directory are typically included using double quotes, e.g., #include "myheader.h", prompting a local directory search first.

Q217 APPLICATION-ORIENTED

Preprocessor Directives & Header Files

A programmer wants to define a constant for the value of pi (3.14159) to use throughout their program. Which is the correct use of a preprocessor directive for this?

A. #include PI 3.14159

B. #define PI 3.14159

C. PI #define 3.14159;

D. const PI #define 3.14159

Ans: B

Explanation:
#define PI 3.14159 is the correct syntax for creating a symbolic constant named PI using the preprocessor, which will be substituted wherever PI appears in the code.

Q218 TRICKY

Preprocessor Directives & Header Files

If a program uses #define SQUARE(x) ((x)*(x)), what does this preprocessor directive create?

A. A simple symbolic constant only

B. A simple macro (with parameters) that performs text substitution to compute the square of x

C. A function that is compiled like a normal C function

D. An error, since #define cannot take parameters

Ans: B

Explanation:
This creates a parameterized macro using #define; the preprocessor substitutes occurrences of SQUARE(x) with the expanded text ((x)*(x)) before compilation.

Q219 STATEMENT-BASED

Preprocessor Directives & Header Files

Statement: 'A header file always contains the actual machine code for the functions it declares.' Is this statement accurate?

A. True

B. False — header files typically contain declarations/prototypes, while actual definitions are often in separate source or library files

C. True, only for stdio.h

D. True, only for user-defined headers

Ans: B

Explanation:
Header files generally provide function declarations, macro definitions, and constants; the actual compiled machine code for standard library functions usually resides in separate library files, not the header itself.

Q220 CONCEPTUAL

Preprocessor Directives & Header Files

Why is it generally good practice to use #include <stdio.h> at the top of nearly every basic C program?

A. It is required even if no input/output functions are used

B. It provides declarations needed for standard input/output functions like printf and scanf, which are commonly used

C. It automatically compiles the program

D. It replaces the need for a main function

Ans: B

Explanation:
Since most basic programs use input/output functions like printf() and scanf(), including stdio.h provides the necessary declarations for these functions to be used correctly.

Q221 FACT-BASED Preprocessor Directives & Header Files

Which preprocessor directive can be used to undo or remove a previously defined macro?

A. #include
B. #undef
C. #ifdef
D. #line

Ans: B

Explanation:
#undef is the preprocessor directive used to remove (undefine) a macro that was previously created using #define.

Q222 PREVIOUS-YEAR PATTERN Preprocessor Directives & Header Files

Which of the following correctly identifies the TWO main preprocessor directives emphasized in the OAVS syllabus?

A. #include and #define
B. #ifdef and #endif
C. #pragma and #error
D. #line and #undef

Ans: A

Explanation:
As per the syllabus, #include (for file inclusion) and #define (for macro/symbolic constant definition) are the two key preprocessor directives covered.

Q223 COMPARISON-BASED Preprocessor Directives & Header Files

How does a macro defined with #define differ fundamentally from a variable declared with a data type like int?

A. A macro is substituted as text by the preprocessor before compilation; a variable is allocated memory and processed during compilation/runtime
B. Both are exactly the same in memory behavior
C. Macros always require a data type just like variables
D. Variables are processed by the preprocessor, not macros

Ans: A

Explanation:
A macro defined via #define undergoes simple textual substitution during preprocessing, with no memory allocation, whereas a variable is allocated actual memory and is processed during compilation and execution.

Q224 OUTPUT-BASED

Preprocessor Directives & Header Files

Given #define LIMIT 100, what will the preprocessor do wherever 'LIMIT' appears in the code before compilation?

A. Leave it unchanged for the compiler to interpret

B. Replace every occurrence of 'LIMIT' with the text '100'

C. Convert 'LIMIT' into a comment

D. Treat 'LIMIT' as a syntax error

Ans: B

Explanation:
The preprocessor performs a direct textual substitution, replacing every occurrence of the macro name 'LIMIT' with its defined value '100' before the compiler processes the code.

Q225 APPLICATIONS

Preprocessor Directives & Header Files

Which of the following is a practical application of header files in modular programming?

A. They prevent any function from being reused

B. They allow declarations of functions/constants to be shared across multiple source files by including the same header

C. They are used only for storing comments

D. They eliminate the need for the main function

Ans: B

Explanation:
Header files enable modular programming by allowing common declarations to be written once and shared across multiple source files through inclusion, promoting consistency and reuse.

Q226 ASSERTION-REASON

Preprocessor Directives & Header Files

Assertion (A): #include <stdio.h> must appear before functions like printf() are used in the program. Reason (R): The preprocessor must insert the function declarations from stdio.h before the compiler encounters their usage. Choose the correct option.

A. Both A and R are true and R correctly explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: A

Explanation:
Both statements are true: the #include directive must appear before usage so that the necessary declarations are available to the compiler when it encounters calls to functions like printf().

Chapter 9: Comments

Q227 DEFINITION-BASED — Comments

What is a 'comment' in a C program?

A. A statement that is executed last

B. Text within the source code that is ignored by the compiler, used to explain or document the code

C. A type of preprocessor directive

D. A mandatory part of every C statement

Ans: B

Explanation:
A comment is explanatory text inserted into source code purely for human readers; the compiler ignores comments entirely during compilation.

Q228 SYNTAX — Comments

Which symbol is used in C to write a single-line comment?

A. /* */

B. //

C. #

D. <!-- -->

Ans: B

Explanation:
Double forward slashes (//) are used to write a single-line comment in C (as per C99 and later, and widely supported), where everything after // on that line is ignored.

Q229 SYNTAX — Comments

Which symbols are used in C to write a multi-line comment?

A. // and //

B. /* and */

C. <!-- and -->

D. # and #

Ans: B

Explanation:
Multi-line comments in C begin with /* and end with */, and everything between these symbols, even across multiple lines, is ignored by the compiler.

Q230 PURPOSE Comments

What is the main purpose of using comments in a C program?

A. To increase the execution speed of the program

B. To improve code readability and documentation by explaining the logic for programmers

C. To replace the need for a header file

D. To define variables

Ans: B

Explanation:
Comments serve to document and explain the code's logic, purpose, or functionality for programmers reading or maintaining the code, without affecting execution.

Q231 CONCEPTUAL Comments

Does the C compiler execute or process the text written inside comments?

A. Yes, comments are always executed

B. No, the compiler ignores comment text entirely; it has no effect on program execution

C. Only single-line comments are ignored

D. Only multi-line comments are ignored

Ans: B

Explanation:
Comments, whether single-line or multi-line, are completely ignored by the compiler and have no impact on the program's logic or execution.

Q232 TRICKY Comments

Can multi-line comments (/* */) be nested inside one another in standard C?

A. Yes, nesting is fully supported

B. No, standard C does not support nested /* */ comments; the first */ encountered ends the comment

C. Yes, but only up to two levels

D. Nesting depends on the variable type used

Ans: B

Explanation:
Standard C does not support nested block comments; the comment ends at the very first */ encountered, regardless of any /* that may appear before it within the intended nested comment.

Q233 OUTPUT-BASED Comments

What will happen if a programmer writes: int x = 5; // declares x \n x = x + 1;?

A. The comment will cause a compilation error

B. Everything after // on that line is ignored, and the next line (x = x + 1;) executes normally

C. The entire program stops at the comment

D. x will not be declared at all

Ans: B

Explanation:
The single-line comment only affects the remainder of that specific line; subsequent lines of code, like x = x + 1;, are compiled and executed normally.

Q234 APPLICATION-ORIENTED Comments

A programmer wants to temporarily disable (comment out) a block of several lines of code for testing. Which comment style is most suitable?

A. // applied to each individual line repeatedly, or /* */ surrounding the whole block

B. Only // can be used for this purpose

C. Comments cannot be used to disable code

D. Only string constants can disable code

Ans: A

Explanation:
A programmer can either prefix every line with // individually, or more conveniently wrap the entire block within a single /* */ pair to comment out multiple lines at once.

Q235 STATEMENT-BASED Comments

Statement: 'Comments can be placed anywhere in a C program, including within or between statements, without affecting program logic.' Is this true?

A. True

B. False

C. True, only at the very beginning of the file

D. True, only inside function bodies

Ans: A

Explanation:
Comments can generally be placed anywhere whitespace is allowed in the code—before, after, or even within parts of statements—without changing the program's logical behavior.

Q236 CONCEPTUAL Comments

Why is excessive or unclear commenting sometimes considered a poor programming practice, despite comments being helpful?

A. Comments always cause runtime errors

B. Comments that are outdated, redundant, or unclear can mislead readers and clutter the code without adding real value

C. The compiler rejects programs with too many comments

D. It is never considered poor practice

Ans: B

Explanation:
While comments aid understanding, excessive, redundant, or outdated comments can clutter code and potentially mislead a reader if they no longer accurately reflect the actual logic.

Q237 FACT-BASED Comments

Which of the following correctly demonstrates valid syntax for a multi-line comment spanning two lines?

A. // This is line one\n// This is line two (two separate single-line comments achieving a similar effect)

B. /* This is line one and this is line two */

C. <comment> text </comment>

D. ## This is a comment ##

Ans: B

Explanation:
/* This is line one and this is line two */ correctly uses the multi-line comment syntax, allowing the comment to span across multiple lines within a single /* */ pair.

Q238 PREVIOUS-YEAR PATTERN Comments

Which TWO comment styles are recognized for use in modern (C99 and later) ANSI-based C compilers, as covered in the syllabus?

A. Single-line (//) and Multi-line (/* */)

B. Hash (#) and Double-hash (##)

C. Angle brackets (< >) only

D. Colon-based (::) comments

Ans: A

Explanation:
The two standard comment styles in C are the single-line comment (//) and the multi-line/block comment (/* */), both used to insert non-executable explanatory text.

Q239 TRICKY Comments

What is a likely consequence of forgetting to close a multi-line comment with */ in a C program?

A. No effect; the compiler automatically closes it at the end of the line

B. All subsequent code until an eventual */ (or end of file) gets swallowed into the comment, potentially causing compilation errors

C. The program runs but ignores only the next single statement

D. It automatically converts to a single-line comment

Ans: B

Explanation:
An unclosed /* comment continues consuming all subsequent code as part of the comment until a closing */ is found (or the file ends), which can unintentionally comment out large portions of code and cause errors.

Q240 CONCEPTUAL Comments

Are comments considered part of the C character set's functional tokens used in actual program logic?

A. Yes, comments are treated as operators

B. No, comments are non-functional text ignored by the compiler and do not contribute to the program's executable logic

C. Yes, comments are treated as identifiers

D. Comments are treated as constants

Ans: B

Explanation:
Comments are purely documentation aids; they are stripped out during compilation and play no role in the actual logic or token stream that the compiler processes for execution.

Chapter 10: sizeof Operator

Q241 DEFINITION-BASED sizeof Operator

What does the 'sizeof' operator do in C?

A. It returns the value stored in a variable

B. It returns the amount of memory (in bytes) occupied by a variable or data type

C. It converts a variable to another data type

D. It deletes a variable from memory

Ans: B

Explanation:
The sizeof operator determines and returns the size, in bytes, that a particular data type or variable occupies in memory.

Q242 SYNTAX sizeof Operator

Which of the following is the correct syntax to use the sizeof operator with a data type?

A. sizeof(int);

B. sizeof int;

C. size_of(int);

D. sizeof[int];

Ans: A

Explanation:
sizeof(int) is the correct syntax, using parentheses around the data type when applying the sizeof operator to a type name.

Q243 USAGE sizeof Operator

Can the sizeof operator be applied directly to a variable without parentheses in C?

A. No, parentheses are always mandatory for any use of sizeof

B. Yes, e.g., sizeof x; is valid when applied to a variable (parentheses optional for variables, though commonly used)

C. No, sizeof can only be used with constants

D. Yes, but only for char variables

Ans: B

Explanation:
When applying sizeof to a variable (an expression), parentheses are technically optional, e.g., sizeof x;, though it's common practice to use them for clarity, as in sizeof(x);.

Q244 RETURN VALUES sizeof Operator

What does the sizeof operator return as its result type?

A. A float value

B. An unsigned integer value (typically of type size_t) representing the size in bytes

C. A character value

D. A string representing the size

Ans: B

Explanation:
The sizeof operator evaluates to an unsigned integer value, conventionally of type size_t, representing the number of bytes occupied by the given type or variable.

Q245 OUTPUT-BASED sizeof Operator

What will sizeof(char) typically return on most standard systems?

A. 1

B. 2

C. 4

D. 8

Ans: A

Explanation:
sizeof(char) typically returns 1, since a char variable is defined to occupy exactly 1 byte of memory in C.

Q246 OUTPUT-BASED sizeof Operator

What will sizeof(int) typically return on most modern 32-bit or 64-bit systems?

A. 2

B. 4

C. 8

D. 1

Ans: B

Explanation:
On most modern 32-bit and 64-bit systems, sizeof(int) typically returns 4, reflecting the common 4-byte size allocated to an int.

Q247 OUTPUT-BASED sizeof Operator

What will sizeof(double) typically return on most standard systems?

A. 4

B. 8

C. 2

D. 16

Ans: B

Explanation:
sizeof(double) typically returns 8, since double-precision floating-point variables commonly occupy 8 bytes of memory.

Q248 APPLICATIONS sizeof Operator

Which of the following is a common practical application of the sizeof operator in C programs?

A. Determining the number of elements in an array by dividing total array size by the size of one element

B. Printing text to the console

C. Comparing two strings for equality

D. Defining a new keyword

Ans: A

Explanation:
A common use of sizeof is calculating the number of elements in an array using the formula: sizeof(array) / sizeof(array[0]), helping write more flexible, portable code.

Q249 DIFFERENCES IN DATA TYPE SIZES sizeof Operator

Why might sizeof(int) return different values (e.g., 2 or 4) on different systems or compilers?

A. sizeof always returns the same fixed value on every system without exception

B. The size of basic data types like int can be implementation-defined and may vary based on the system architecture and compiler

C. sizeof only works on 32-bit systems

D. It depends on the variable's name length

Ans: B

Explanation:
The C standard does not mandate an exact fixed size for int; the actual size is implementation-defined and can vary based on the underlying system architecture and compiler design.

Q250 CONCEPTUAL sizeof Operator

Is sizeof evaluated at compile-time or runtime for fixed data types like int, float, char?

A. Always at runtime, never at compile-time

B. Generally at compile-time, since the size of standard types is typically known to the compiler in advance

C. Only during the linking phase

D. Only when the program is loaded into memory

Ans: B

Explanation:
For standard fixed-size data types, sizeof is generally evaluated at compile-time, since the compiler already knows the storage requirements of these types in advance.

Q251 TRICKY sizeof Operator

What is the result of sizeof('A') in standard C, considering how character constants are treated?

A. Always 1, same as sizeof(char)

B. In standard C, character constants like 'A' are treated as type int, so sizeof('A') often equals sizeof(int), not sizeof(char)

C. It always causes a compilation error

D. It returns 0

Ans: B

Explanation:
In C (unlike C++), character constants are promoted to type int, so sizeof('A') commonly equals sizeof(int) (e.g., 4 on many systems), which can surprise programmers expecting 1.

Q252 OUTPUT-BASED sizeof Operator

Given int arr[5]; and assuming sizeof(int) is 4, what will sizeof(arr) return?

A. 5

B. 4

C. 20

D. 1

Ans: C

Explanation:
sizeof(arr) for an array returns the total memory occupied by all its elements; with 5 integers of 4 bytes each, this equals 5 × 4 = 20 bytes.

Q253 APPLICATION-ORIENTED sizeof Operator

A programmer wants to dynamically check how much memory a 'long int' occupies on their specific system without looking it up manually. Which operator should they use?

A. typeof
B. sizeof
C. memof
D. lengthof

Ans: B

Explanation:
The sizeof operator is specifically designed to determine, at compile-time, how much memory a data type or variable occupies on the current system.

Q254 COMPARISON-BASED sizeof Operator

How does sizeof(float) generally compare to sizeof(double) in terms of returned byte size?

A. sizeof(float) is typically smaller (e.g., 4 bytes) compared to sizeof(double) (typically 8 bytes)
B. Both always return exactly the same value
C. sizeof(float) is always larger than sizeof(double)
D. Neither can be measured using sizeof

Ans: A

Explanation:
float is generally a single-precision type occupying 4 bytes, while double is double-precision, typically occupying 8 bytes, reflecting their differing storage and precision capacities.

Q255 STATEMENT-BASED sizeof Operator

Statement: 'The sizeof operator can be used with both data types and variables/expressions in C.' Is this correct?

A. True
B. False, sizeof works only with data types
C. False, sizeof works only with variables
D. True, but only inside loops

Ans: A

Explanation:
sizeof is versatile and can be applied both to type names (like sizeof(int)) and to variables or expressions (like sizeof(x) or sizeof(x + y)).

Q256 FILL-IN-THE-BLANK sizeof Operator

The sizeof operator returns the size of its operand in _____.

A. Bits

B. Bytes

C. Kilobytes

D. Characters only

Ans: B

Explanation:
The sizeof operator measures and returns the storage size of its operand specifically in bytes, the standard unit for memory measurement in C.

Q257 PREVIOUS-YEAR PATTERN sizeof Operator

Which of the following best summarizes the primary role of the sizeof operator as covered in the OAVS syllabus?

A. To determine the memory (in bytes) occupied by a data type or variable, useful for portability and memory management

B. To perform string concatenation

C. To convert between data types permanently

D. To compare two variables for equality

Ans: A

Explanation:
As emphasized in the syllabus, sizeof's primary role is to determine the storage size of data types or variables in bytes, aiding portability and efficient memory usage.

Q258 ASSERTION-REASON sizeof Operator

Assertion (A): sizeof is classified as an operator, not a function, in C. Reason (R): Despite its function-like syntax with parentheses, sizeof is evaluated by the compiler itself as part of the language's operator set. Choose the correct option.

A. Both A and R are true and R correctly explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: A

Explanation:
Both statements are true: although sizeof is often written with parentheses resembling a function call, it is technically a compile-time operator built into the C language, not a callable function.

Q259 TRICKY sizeof Operator

Why is the result type of sizeof important when used in arithmetic comparisons involving signed integers?

A. sizeof never returns a meaningful value

B. Since sizeof returns an unsigned value (size_t), comparisons with signed negative numbers can lead to unexpected results due to implicit type conversion

C. sizeof always returns a negative number

D. It has no bearing on comparisons whatsoever

Ans: B

Explanation:

Because sizeof yields an unsigned value, comparing it directly with a signed negative number can cause the signed value to be implicitly converted to unsigned, potentially producing unexpected or incorrect comparison results.

Chapter 11: Steps Involved in Translation of a C Program

Q260 DEFINITION-BASED — Translation of C Program

What does 'translation' of a C program refer to?

A. The process of converting C code into another programming language like Python

B. The complete process of converting C source code into an executable machine-readable program through several stages

C. Renaming variables in the program

D. Writing comments in multiple languages

Ans: B

Explanation:
Translation refers to the overall process by which human-readable C source code is converted, through several defined stages, into an executable program that the machine can run.

Q261 SEQUENCE OF TRANSLATION PHASES — Translation of C Program

What is the correct sequence of the major steps involved in translating a C program, from writing the code to execution?

A. Editing, Preprocessing, Compilation, Linking, Loading, Execution

B. Compilation, Editing, Linking, Execution, Loading, Preprocessing

C. Execution, Loading, Linking, Compilation, Preprocessing, Editing

D. Linking, Compilation, Preprocessing, Editing, Loading, Execution

Ans: A

Explanation:
The standard sequence is: Editing (writing code), Preprocessing (handling directives), Compilation (translating to object code), Linking (combining with libraries), Loading (placing into memory), and finally Execution (running the program).

Q262 DEFINITION-BASED Translation of C Program

What happens during the 'Editing' phase of C program translation?

A. The compiler converts code to machine language

B. The programmer writes and modifies the C source code using a text editor or IDE

C. Libraries are linked to the object file

D. The program is loaded into RAM for execution

Ans: B

Explanation:
The editing phase is where the programmer writes, modifies, and saves the C source code, typically using a text editor or an Integrated Development Environment (IDE).

Q263 DEFINITION-BASED Translation of C Program

What is the primary role of the 'Preprocessing' phase?

A. Converting source code directly into an executable file

B. Processing preprocessor directives such as #include and #define before actual compilation

C. Allocating memory for variables at runtime

D. Linking external libraries

Ans: B

Explanation:
During preprocessing, directives like #include and #define are processed, expanding header files and substituting macros, producing an expanded source code ready for compilation.

Q264 DEFINITION-BASED Translation of C Program

What occurs during the 'Compilation' phase of C program translation?

A. The source/expanded code is translated into object code (typically machine or assembly-level code)

B. The program's output is displayed on the screen

C. Variables are renamed automatically

D. The program is saved permanently to disk as a text file

Ans: A

Explanation:
Compilation translates the preprocessed source code into object code, which is a low-level representation closer to machine language but not yet a complete executable.

Q265 DEFINITION-BASED Translation of C Program

What is the primary function of the 'Linking' phase?

A. Writing comments into the code

B. Combining the object code of the program with required library functions and other object files to produce a complete executable

C. Displaying error messages only

D. Converting variables into constants

Ans: B

Explanation:
Linking combines the program's own object code with the object code of library functions (like printf from stdio) and any other necessary modules, producing a single executable file.

Q266 DEFINITION-BASED Translation of C Program

What happens during the 'Loading' phase of program translation?

A. The source code is first written by the programmer

B. The executable file is loaded from secondary storage (like a hard disk) into the computer's main memory (RAM) for execution

C. Errors in syntax are detected

D. Header files are expanded

Ans: B

Explanation:
Loading involves transferring the executable program from storage into the main memory (RAM), preparing it to be run by the processor.

Q267 DEFINITION-BASED Translation of C Program

What occurs during the final 'Execution' phase?

A. The source code is edited for the first time

B. The loaded program's instructions are actually carried out by the CPU, producing the program's output

C. The object files are merged with libraries

D. Preprocessor directives are expanded

Ans: B

Explanation:
Execution is the final stage where the CPU actually carries out the instructions of the loaded program, performing computations and producing the intended output.

Q268 CONCEPTUAL Translation of C Program

Why must preprocessing occur BEFORE compilation in the translation process?

A. Preprocessing has no real dependency on compilation order

B. Because directives like #include and #define must be expanded/substituted first so the compiler works with the complete, expanded source code

C. Because compilation happens before the code is even written

D. Because linking requires preprocessing to happen afterward

Ans: B

Explanation:
Since the compiler needs the fully expanded code (with headers included and macros substituted) to correctly translate it, preprocessing must logically occur before compilation begins.

Q269 CONCEPTUAL Translation of C Program

Why is 'Linking' necessary even after a program has been successfully compiled into object code?

A. Linking is never actually necessary

B. Because the object code often references external library functions (e.g., printf) whose actual code resides elsewhere and must be combined to form a complete executable

C. Linking only renames the executable file

D. Linking happens before compilation, not after

Ans: B

Explanation:
A compiled object file frequently contains calls to library functions whose implementations exist separately; linking resolves these references by combining all necessary object code into one complete executable.

Q270 COMPARISON-BASED Translation of C Program

What is the key difference between 'Compilation' and 'Linking' in the translation process?

A. Compilation translates source code to object code; linking combines object code with libraries to form an executable

B. They are identical processes with different names

C. Linking happens before compilation always

D. Compilation only checks for spelling errors

Ans: A

Explanation:
Compilation focuses on translating the program's own source code into object code, while linking's role is to combine that object code with external library code to produce a single runnable executable.

Q271 COMPARISON-BASED Translation of C Program

What is the key difference between 'Loading' and 'Execution'?

A. Loading places the executable into memory; execution is the actual running/processing of the loaded instructions by the CPU

B. They occur in reverse order

C. Loading and execution are the same step

D. Loading happens only after execution completes

Ans: A

Explanation:
Loading is the preparatory step of bringing the executable into RAM, while execution is the subsequent step where the CPU actually processes and runs those loaded instructions.

Q272 OUTPUT-BASED Translation of C Program

If a C program has a syntax error (e.g., a missing semicolon), at which phase would this typically be detected?

A. Editing phase, automatically by the text editor without any tool

B. Compilation phase, when the compiler attempts to translate the code and encounters the invalid syntax

C. Loading phase only

D. Execution phase only, never earlier

Ans: B

Explanation:
Syntax errors, such as a missing semicolon, are typically detected during the compilation phase when the compiler parses the code and finds it does not conform to the language's grammar rules.

Q273 OUTPUT-BASED Translation of C Program

If a program calls a library function like printf() but a required library file is missing during the build process, at which phase would an error most likely occur?

A. Editing phase

B. Preprocessing phase

C. Linking phase, due to an unresolved external reference

D. Loading phase only

Ans: C

Explanation:
Since linking is responsible for resolving references to external library functions, a missing library would typically cause a 'linker error' during the linking phase, such as an unresolved symbol/reference.

Q274 APPLICATION-ORIENTED Translation of C Program

A student writes #include <stdio.h> at the top of their program. During which translation phase is this specific line actually processed and expanded?

A. Execution phase

B. Preprocessing phase

C. Loading phase

D. Linking phase

Ans: B

Explanation:
The #include directive is a preprocessor directive, and its expansion (inserting the content of stdio.h) occurs specifically during the preprocessing phase, before compilation.

Q275 TRICKY Translation of C Program

Can a C program skip the 'Linking' phase entirely if it uses absolutely no external library functions?

A. No, linking is always mandatory regardless of content

B. In practice, even minimal programs typically still require some linking (e.g., for runtime startup code), though the linking step conceptually still applies as part of the standard translation process

C. Yes, linking is completely unnecessary for any C program

D. Linking only applies to C++ programs, not C

Ans: B

Explanation:
Even seemingly simple programs generally require linking with startup/runtime code provided by the compiler/OS, so linking remains a standard part of the translation process even when explicit library calls seem minimal.

Q276 STATEMENT-BASED Translation of C Program

Statement: 'The six phases of translation (Editing, Preprocessing, Compilation, Linking, Loading, Execution) always occur in a fixed sequential order for a standard C program build-and-run cycle.' Is this generally true?

A. True

B. False, they occur in random order

C. True, but Execution always happens first

D. False, Linking always happens before Compilation

Ans: A

Explanation:
For a standard build-and-run cycle, these six phases generally do occur in the described sequential order: Editing, Preprocessing, Compilation, Linking, Loading, and finally Execution.

Q277 FILL-IN-THE-BLANK Translation of C Program

The phase in which the source code is translated into object code by the compiler is called the _____ phase.

A. Editing

B. Compilation

C. Loading

D. Execution

Ans: B

Explanation:
The Compilation phase is specifically where the compiler translates the (preprocessed) source code into object code.

Q278 PREVIOUS-YEAR PATTERN Translation of C Program

Which of the following correctly lists the SIX standard phases of C program translation as per the OAVS syllabus, in proper order?

A. Editing, Preprocessing, Compilation, Linking, Loading, Execution

B. Preprocessing, Editing, Linking, Compilation, Execution, Loading

C. Compilation, Linking, Editing, Loading, Preprocessing, Execution

D. Loading, Execution, Editing, Compilation, Linking, Preprocessing

Ans: A

Explanation:
The correct and standard order, as specified in the syllabus, is Editing, Preprocessing, Compilation, Linking, Loading, and Execution.

Q279 CONCEPTUAL Translation of C Program

Why is the 'Editing' phase considered the very first step in the translation process?

A. Because the CPU executes editing instructions first

B. Because the source code must first be written/ created by the programmer before any subsequent processing (preprocessing, compiling, etc.) can occur

C. Because editing happens after compilation

D. Because editing is the same as execution

Ans: B

Explanation:
Since all subsequent phases (preprocessing, compilation, etc.) operate on the source code, the program must first be written during the editing phase before any translation can begin.

Q280 ASSERTION-REASON Translation of C Program

Assertion (A): An 'object file' produced after compilation is generally not directly executable on its own. Reason (R): Object files often contain unresolved references to external functions that must be resolved through linking before becoming a complete executable. Choose the correct option.

A. Both A and R are true and R correctly explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: A

Explanation:
Both statements are true and connected: object files frequently contain calls to external functions not yet resolved, which is precisely why linking is required before the program becomes a complete, executable file.

Q281 APPLICATION-ORIENTED Translation of C Program

When a user double-clicks on an already-compiled and linked executable file to run a C program, which phases are primarily being invoked at that point?

A. Editing and Preprocessing

B. Loading and Execution

C. Compilation and Linking

D. Only Editing

Ans: B

Explanation:
Running an already-built executable primarily involves Loading (bringing the program into memory) followed by Execution (the CPU running its instructions); editing, preprocessing, compiling, and linking have already been completed beforehand.

Q282 TRICKY Translation of C Program

If a C program compiles successfully with no errors but produces incorrect output when run, during which phase would this logical error most likely have originated, even though it surfaces during execution?

A. It must have originated during linking, not earlier

B. It most likely originated during the editing phase, where the programmer wrote flawed logic, even though the incorrect behavior becomes visible only during execution

C. It must have originated during loading

D. It must have originated during preprocessing only

Ans: B

Explanation:
Logical errors (as opposed to syntax errors) typically originate from flawed code written during the editing phase; such errors often pass compilation and linking successfully but reveal incorrect behavior only when the program is actually executed.